Study of Convolutional Neural Networks, Transfer Learning, and Visual Interpretability

Programming Assignment V

CS601T: Deep Learning

Group 30

Submitted by

Anushaa B (CS24BT032)
Skanda P (CS24BT045)
Thiaana B (CS24BT031)

Under the Supervision of

Prof. Dileep A.D.

Department of Computer Science and Engineering
Indian Institute of Technology Dharwad

November 2026

Abstract

This report presents an empirical and theoretical investigation of Convolutional Neural Networks (CNNs), regularization dynamics, transfer learning paradigms, and visual explainability techniques. The experimental work is organized into two primary tracks:

Part A: CNN Mechanics and Digit Classification on MNIST. We deconstruct the convolutional pipeline on a five-class MNIST subset (digits 3, 4, 5, 7, and 8) comprising 11,385 training, 3,795 validation, and 3,795 test images (60/20/20 split). We evaluate an isolated 5×5 Kaiming-initialized kernel, demonstrating that spatial cross-correlation produces directional stroke responses while verifying exact equivalence to manual sliding-window traversal (maximum absolute difference 3.43×10^{-7}, well within the 10^{-5} tolerance). We construct a two-stage hierarchical feature extractor (Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool), tracing spatial contraction from 28×28 to 4×4 and logging tensor slices to analyze non-linear thresholding and spatial downsampling ($\approx 51\%$ sparsity after ReLU1). Finally, an end-to-end classifier trained with mini-batch SGD achieves **98.02%** test accuracy with minimal overfitting (generalization gap 0.87 percentage points). We show that learned Conv1 filters organize into oriented edge detectors, compute the exact 14×14 receptive field for Conv2 units, and demonstrate that top-activating neurons localize on class-defining strokes with channel-level specialization (e.g., Channel 12 dominating digit 4, while digit 7 utilizes a combination of channels 3, 12, 6, and 16).

Part B: Custom CNNs, Dropout Regularization, and VGG-19 Transfer Learning. Addressing extreme data scarcity (250 training images across five Caltech-101 categories: *butterfly*, *car_side*, *grand_piano*, *scorpion*, and *starfish*), we train nine custom CNN models spanning three architectural depths (4-layer, 5-layer, and 6-layer) under dropout rates $p \in \{0.0, 0.3, 0.5\}$. All custom models exhibit overfitting (training accuracy $\geq 91.6\%$ vs. validation accuracy $\leq 90.0\%$), but dropout contracts the mean generalization gap on average from 18.4 down to 11.5 percentage points. While Architecture 1 at $p = 0.3$ is selected as the optimal model under our validation policy (Validation Accuracy: **90.0%**, Validation Loss: 0.4849, Test Accuracy: **73.0%**), the parameter-efficient Architecture 2 (48,405 parameters vs. 211,701 in Arch 1) attains the highest test accuracy across every dropout rate (80.0%, 83.0%, and 81.0%). In contrast, feature extraction using pre-trained VGG-19 with a frozen ImageNet backbone and a trainable 5-way classification head (20,485 trainable parameters, without batch-level normalization) overcomes data scarcity, achieving **99.0%** test accuracy with only 1 error on the test set and 0 errors on the difficult *scorpion* class.

Visual Interpretability and Attribution. We calculate and verify theoretical receptive fields (51×51 in Arch 1, 99×99 in Arch 2, and 252×252 in VGG-19, clipped to 224×224). Guided Backpropagation isolates boundary patterns that excite the top-5 feature neurons on the representative butterfly image. Furthermore, Grad-CAM class activation mapping produces focused localization heatmaps for ground-truth classes while assigning near-zero Softmax probabilities ($p \approx 0.000$) under counterfactual class queries.

Contents

Abstract i

1 Introduction and Theoretical Foundations 1

1.1 Continuous vs. Discrete Convolution and Cross-Correlation . . . 1
1.2 Spatial Transformation: Stride, Padding, and Output Dimensions . . . 2
1.3 Inductive Biases of CNNs . . . 2
1.4 Pooling and Spatial Invariance . . . 3
1.5 Theory of Receptive Fields . . . 3
1.6 Weight Initialization and Training Dynamics . . . 3
1.7 Loss Formulation and Inference Probabilities . . . 4
1.8 Regularization via Dropout . . . 4
1.9 Transfer Learning Paradigm . . . 4
1.10 Visual Interpretability Methodologies . . . 5

2 Experimental Setup and Reproducibility 6

2.1 Datasets and Data Partitions . . . 6
2.1.1 Part A: Five-Class MNIST Subset . . . 6
2.1.2 Part B: Five-Class Caltech-101 Visual Subset . . . 7

3 Part A: Feature Extraction and Classification on MNIST 8

3.1 Part A Q1: Isolated Single Filter Convolution Analysis . . . 8
3.1.1 Kernel Specification and Spatial Dimension Verification . . . 8
3.1.2 Manual Sliding-Window vs. Framework Implementation . . . 9
3.1.3 Feature Map Visualizations Across All Digits . . . 9
3.1.4 Observation . . . 10
3.2 Part A Q2: Two-Stage Hierarchical Feature Extraction . . . 11
3.2.1 Architectural Hierarchy and Shape Verification . . . 11
3.2.2 Filter Visualizations . . . 11
3.2.3 Feature Map Progression and Tensor Logging . . . 12
3.2.4 Observation . . . 13
3.3 Part A Q3a: Supervised Classification on MNIST Subset . . . 13
3.3.1 Model Architecture and Hyperparameter Configuration . . . 13

3.3.2 Quantitative Results and Training Dynamics . . . 13
3.3.3 Observation . . . 14
3.4 Part A Q3b: Learned Convolutional Filters and Feature Representations . . . 15
3.4.1 Filter Architecture Clarification . . . 15
3.4.2 Observation . . . 16
3.5 Part A Q3c: Maximum Activation Neurons and Receptive Field Mapping . . . 17
3.5.1 Theoretical Receptive Field Derivation . . . 17
3.5.2 Top-5 Activating Neurons and Channel Indexing . . . 17
3.5.3 Observation . . . 19

4 Part B (Q1): Custom CNN Architectures and Dropout Study on Caltech-101 21
4.1 Part B Q1a: Custom CNN Architectural Exploration . . . 21
4.1.1 Architectural Configurations and Dimension Verification . . . 21
4.1.2 Training Protocol and Optimization . . . 23
4.2 Part B Q1b: Hyperparameter Search and Dropout Regularization Study . . . 23
4.2.1 Comprehensive Results Across All Nine Configurations . . . 23
4.2.2 Observations on Regularization, Depth, and Error Distributions . . . 25
4.3 Part B Q1c: Feature Representations in Best Custom CNN . . . 27
4.3.1 Observation . . . 27
4.4 Part B Q1d: Receptive Field Mapping and Bounding Box Analysis . . . 27
4.4.1 Mathematical Derivation of Receptive Fields . . . 27
4.4.2 Empirical Receptive Field Patches . . . 28
4.4.3 Observation . . . 30

5 Part B (Q2): Transfer Learning with VGG-19 and Explainability 31
5.1 Part B Q2a: Transfer Learning with Pretrained VGG-19 . . . 31
5.1.1 Network Architecture and Feature Extraction . . . 31
5.1.2 Training Protocol and Early Stopping . . . 31
5.1.3 Classification Performance and Error Analysis . . . 32
5.1.4 Observation . . . 34
5.2 Part B Q2b: Deep Receptive Field Analysis at `block5_conv4` . . . 34
5.2.1 Theoretical Derivation at `block5_conv4` . . . 34
5.2.2 Empirical Receptive Field Coordinates . . . 34
5.3 Part B Q2c: Guided Backpropagation for Feature Attribution . . . 36
5.3.1 Top-5 Neurons on Butterfly Image . . . 36
5.3.2 Observation . . . 38
5.4 Part B Q2d: Class Activation Mapping with Grad-CAM . . . 38
5.4.1 Full 5×5 Cross-Class Query Matrix . . . 38
5.4.2 Observation . . . 42

6 Synthesis, Comparative Discussion, and Conclusion **43**

6.1 Synthesis of Findings 43

6.2 Study Limitations 44

6.3 Concluding Remarks 44

References **45**

List of Figures

3.1 Single 5×5 convolution filter analysis: centered kernel weights followed by input images and corresponding feature maps for digits 3, 4, 5, 7, and 8, stacked vertically. 10
3.2 Selected untrained filters from Conv1 ($1 \times 5 \times 5$) and Conv2 ($6 \times 5 \times 5$, displayed as channel-mean). 12
3.3 Feature maps across Stage 1 and Stage 2 for representative digit 3 (showing post-ReLU activations before pooling). 12
3.4 Training and validation loss and accuracy curves for digit classifier over 15 epochs. 14
3.5 Confusion matrices on MNIST training, validation, and test splits (Classes: 3, 4, 5, 7, 8). 14
3.6 Learned convolutional filters from the best-validation checkpoint, shown in separate enlarged panels. 16
3.7 Feature map representations of representative digit 3 generated by the trained network. 16
3.8 Receptive field bounding boxes (14×14 pixels) for top-5 activating neurons overlaid on input digits. 19

4.1 Comparative grouped bar chart of training, validation, and test accuracies for all 9 custom CNN models. 24
4.2 Training loss and validation accuracy curves for custom CNN architectures under dropout $p = 0.3$. 24
4.3 Complete confusion matrix evaluation for custom CNN architectures under dropout $p = 0.3$ across train, validation, and test splits. 25
4.4 Conv1 (54×54) and Conv2 (22×22) post-ReLU feature maps from the optimal Custom CNN (Arch 1, $p = 0.3$) for a representative butterfly training image. . . 27
4.5 Receptive field bounding boxes (51×51 pixels) for maximum-activation neurons in the optimal Custom CNN. 29

5.1 Training curves and comprehensive confusion matrix evaluation for VGG-19 feature extraction. 33
5.2 Side-by-side comparison of test confusion matrices: Best Custom CNN (Arch 1, $p = 0.3$, 73.0%) vs. Pre-trained VGG-19 (99.0%). 33

5.3 Receptive field bounding boxes at `block5_conv4` of VGG-19 (252×252, clipped to image dimensions). 35
5.4 Guided Backpropagation feature attribution across the top-5 neurons for the representative butterfly image at `block5_conv4`. 36
5.5 Supplementary Guided Backpropagation visualizations for the top-activating neuron per class. 37
5.6 Complete 5×5 Grad-CAM localization matrix: Rows correspond to input images, columns to queried target classes, with Softmax probabilities indicated above each panel. 39
5.7 Enlarged on-target Grad-CAM heatmaps: Target class matches ground truth ($p \geq 0.999$ for four classes, 0.957 for starfish). 40
5.8 Counterfactual Grad-CAM heatmaps evaluating a butterfly input against non-target class hypotheses. 41

List of Tables

2.1 MNIST Five-Class Subset Split Statistics (Part A). 7
2.2 Caltech-101 Five-Category Dataset Split Statistics (Part B). 7

3.1 Single 5×5 Kaiming-Initialized Kernel Weights (Seed 42). 8
3.2 Layer-by-Layer Shape Verification in Two-Stage Stacked Feature Extractor. . . . 11
3.3 Classification Performance of Trained Digit Classifier on MNIST Subset. 14
3.4 Top-5 Activating Neurons at Conv2 Output Across Five Representative MNIST Digits. 18

4.1 Layer-by-Layer Feature Map Shapes Across Custom CNN Architectures. 22
4.2 Architectural Comparison and Parameter Distribution Across Custom CNNs. . . 23
4.3 Performance Metrics Across All 9 Custom CNN Models on Caltech-101 Subset. . 24
4.4 Maximum-Activation Neurons and 51×51 Receptive Field Coordinates in Optimal Custom CNN (Arch 1, $p = 0.3$). 28

5.1 Classification Performance of Pre-trained VGG-19 Feature Extractor on Caltech-101 Subset. 32
5.2 Maximum-Activation Neurons and Bounding Boxes at `block5_conv4` in VGG-19. 35
5.3 Top-5 Activating Neurons at `block5_conv4` on the Representative Butterfly Image. 36

Chapter 1

Introduction and Theoretical Foundations

Convolutional Neural Networks (CNNs) represent a foundational architectural paradigm in deep learning and computer vision [2, 3]. Traditional Multi-Layer Perceptrons (MLPs) treat input data as flat vectors, disregarding the spatial topology inherent in visual imagery. When applied to high-dimensional images, MLPs suffer from substantial parameter counts, lack translation equivariance, and fail to exploit local statistical stationarity. CNNs address these limitations through three primary structural inductive biases: **sparse connectivity**, **weight sharing**, and **spatial subsampling (pooling)** [11].

This chapter outlines the formal mathematical foundations governing spatial convolution, receptive field expansion, regularization via dropout, weight initialization, transfer learning, and gradient-based interpretability, drawing directly upon the theoretical principles developed in CS601T [1].

1.1 Continuous vs. Discrete Convolution and Cross-Correlation

In mathematical analysis, continuous convolution between two continuous functions $f(t)$ and $g(t)$ is defined as the integral of the product of the two functions after one is reflected across the origin and translated [11]:

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t-\tau)d\tau \tag{1.1}$$

For discrete two-dimensional signals such as an image $I \in \mathbb{R}^{H \times W}$ and a kernel $K \in \mathbb{R}^{k_h \times k_w}$, the 2D discrete convolution is given by:

$$(I * K)(i,j) = \sum_m \sum_n I(m,n)K(i-m, j-n) \tag{1.2}$$

In computational deep learning frameworks such as PyTorch, the kernel is not flipped prior to sliding. Instead, the operation implemented is mathematically **cross-correlation**:

$$S(i,j) = (I \star K)(i,j) = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} I(i+m, j+n) K(m,n) \tag{1.3}$$

Because the kernel weights $K(m,n)$ are learnable parameters optimized via gradient descent, reflection is functionally redundant; a network learning cross-correlation simply learns the reflected version of a true convolutional kernel with equivalent expressive capacity.

1.2 Spatial Transformation: Stride, Padding, and Output Dimensions

Let an input feature map have height H_{in} and width W_{in}, convolved with a kernel of spatial dimensions $K_h \times K_w$, with padding P_h, P_w applied to the boundaries and spatial stride S_h, S_w. The resulting spatial dimensions $H_{out} \times W_{out}$ of the output feature map are given by the discrete integer division formula [1]:

$$H_{out} = \left\lfloor \frac{H_{in} - K_h + 2P_h}{S_h} \right\rfloor + 1, \quad W_{out} = \left\lfloor \frac{W_{in} - K_w + 2P_w}{S_w} \right\rfloor + 1 \tag{1.4}$$

When multiple input channels C_{in} are present, each output channel $c_{out} \in \{1, \ldots, C_{out}\}$ is computed by summing across independent 2D convolutions over all input channels plus a scalar bias $b_{c_{out}}$:

$$Y_{c_{out}}(i,j) = b_{c_{out}} + \sum_{c_{in}=1}^{C_{in}} \sum_{m=0}^{K_h-1} \sum_{n=0}^{K_w-1} X_{c_{in}}(i \cdot S_h + m, j \cdot S_w + n) W_{c_{out}, c_{in}}(m,n) \tag{1.5}$$

1.3 Inductive Biases of CNNs

CNNs incorporate three structural priors that render them well suited for visual data:

1. **Sparse Connectivity:** Unlike dense layers where every input unit connects to every output unit, each convolutional neuron connects only to a localized spatial patch of the input, reducing parameters and focusing capacity on local pixel correlations.

2. **Weight Sharing:** The same kernel parameters W are reused across the entire spatial extent of the input. This embodies the physical prior that low-level features (e.g., edges, corners, textures) are statistically stationary and useful across different image locations.

3. **Translation Equivariance:** A function f is equivariant to a transformation g if $f(g(x)) = g(f(x))$. Convolution is equivariant to spatial translations: shifting an input image shifts the resulting feature map by the corresponding stride-scaled displacement.

1.4 Pooling and Spatial Invariance

Pooling operations (most commonly Max-Pooling) replace localized feature neighborhoods with summary statistics:

$$Y_{\text{pool}}(i,j) = \max_{0 \le m < K_p, 0 \le n < K_p} X(i \cdot S_p + m, j \cdot S_p + n) \tag{1.6}$$

where K_p denotes the pooling window size and S_p denotes the pooling stride (differentiated from padding parameters P_h, P_w). Pooling downsamples spatial resolution, progressively reducing computational memory footprint, and introduces local translation invariance within the pooling window.

1.5 Theory of Receptive Fields

The **Receptive Field** (RF) of a neuron at layer l denotes the spatial extent (measured in pixels) of the input image that influences that neuron's activation [1].

Let $RF_0 = 1$, cumulative stride (jump) $J_0 = 1$, and receptive field center coordinate start $start_0 = 0.5$ (measuring pixel centers from 0). For any subsequent layer l with kernel size k_l, stride s_l, and padding p_l, the recursive expansion implemented in the code is:

$$RF_l = RF_{l-1} + (k_l - 1) \cdot J_{l-1} \tag{1.7}$$

$$start_l = start_{l-1} + \left(\frac{k_l - 1}{2} - p_l \right) \cdot J_{l-1} \tag{1.8}$$

$$J_l = J_{l-1} \cdot s_l \tag{1.9}$$

For a neuron located at spatial coordinates (i, j) (0-indexed row i, column j) in the feature map of layer l, the center of its receptive field (c_y, c_x) in the input image is:

$$c_y = start_l + i \cdot J_l, \quad c_x = start_l + j \cdot J_l \tag{1.10}$$

The corresponding physical bounding box coordinates in the input image $[y_1, x_1, y_2, x_2]$ are given by:

$$\begin{aligned} y_1 &= \max\left(0, \lfloor c_y - RF_l/2 \rfloor\right), & x_1 &= \max\left(0, \lfloor c_x - RF_l/2 \rfloor\right) \\ y_2 &= \min\left(H, \lceil c_y + RF_l/2 \rceil\right), & x_2 &= \min\left(W, \lceil c_x + RF_l/2 \rceil\right) \end{aligned} \tag{1.11}$$

where H and W denote the height and width of the input image.

1.6 Weight Initialization and Training Dynamics

To maintain stable activation and gradient variances during backpropagation across deep networks, layer weights are initialized based on their fan-in and fan-out dimensions:

- **Kaiming (He) Initialization [5]:** Designed for layers followed by Rectified Linear Units (ReLU). In Part A, convolutional layers use Kaiming normal initialization with `mode="fan_in"` and ReLU gain, sampling weights from $\mathcal{N}(0, \sigma^2)$ where $\sigma = \sqrt{2/\text{fan_in}} = \sqrt{2/(C_{in}K_hK_w)}$, with biases initialized to zero. In Part B custom CNNs, convolutional layers are initialized with Kaiming normal using `mode="fan_out"` ($\sigma = \sqrt{2/\text{fan_out}}$) and zero biases.

- **Xavier (Glorot) Initialization [6]:** Applied to linear (fully connected) layers across both Part A and Part B. Weights are sampled from a uniform distribution $\mathcal{U}(-a, +a)$ with bound $a = \sqrt{6/(\text{fan_in} + \text{fan_out})}$, with biases initialized to zero.

- **Optimization Dynamics:** Part A employs mini-batch Stochastic Gradient Descent (SGD) with momentum 0. Part B employs the Adam optimizer [8] with decoupled weight decay.

1.7 Loss Formulation and Inference Probabilities

For multi-class classification over C classes, the network outputs unnormalized logits $\mathbf{z} \in \mathbb{R}^C$. Training employs the Cross-Entropy Loss with numerically stable log-sum-exp accumulation:

$$\mathcal{L}_{\text{CE}}(\mathbf{z}, y) = -\log\left(\frac{\exp(z_y)}{\sum_{j=1}^{C}\exp(z_j)}\right) = -z_y + \log\left(\sum_{j=1}^{C}\exp(z_j)\right) \tag{1.12}$$

At evaluation and inference time, normalized class posterior probabilities $\mathbf{p} = \text{Softmax}(\mathbf{z})$ are computed:

$$p_c = \frac{\exp(z_c)}{\sum_{j=1}^{C}\exp(z_j)} \tag{1.13}$$

1.8 Regularization via Dropout

Dropout [7] is a stochastic regularization technique where individual neuron activations are set to zero with probability p during training:

$$r_j \sim \text{Bernoulli}(1-p), \quad \widetilde{y}_j = \frac{r_j}{1-p}y_j \tag{1.14}$$

The scaling factor $\frac{1}{1-p}$ (inverted dropout) ensures that the expected activation magnitude remains invariant between training and evaluation phases.

1.9 Transfer Learning Paradigm

When training deep neural networks on small datasets, training from scratch frequently leads to overfitting [3, 4]. Transfer learning addresses this by repurposing weights pre-trained on

large-scale datasets such as ImageNet (1.2 million images across 1,000 categories). In feature-extraction transfer learning, the pre-trained convolutional backbone is kept frozen ($\nabla_{\theta_{conv}} \mathcal{L} = 0$), preserving generic visual representations (edges, textures, shapes), while a lightweight custom classification head is trained on the target domain.

1.10 Visual Interpretability Methodologies

To analyze the internal decision-making process of deep CNNs, two gradient-based attribution methods are employed:

1. **Guided Backpropagation [9]:** Guided backprop combines deconvolution and backpropagation by passing gradients only through paths where *both* the forward activation and the backward gradient are strictly positive:

$$R_i^l = (R_i^{l+1} > 0) \cdot (f_i^l > 0) \cdot R_i^{l+1} \tag{1.15}$$

   This suppresses negative gradients, yielding high-resolution pixel-space visualizations of the input patterns that excite a target neuron.

2. **Grad-CAM (Gradient-Weighted Class Activation Mapping) [10]:** Grad-CAM calculates the gradient of the target class logit y^c with respect to feature maps A^k of the final convolutional layer. These gradients are globally average-pooled to capture channel importance weights α_k^c:

$$\alpha_k^c = \frac{1}{Z} \sum_{i=1}^{U} \sum_{j=1}^{V} \frac{\partial y^c}{\partial A_{ij}^k} \tag{1.16}$$

   A weighted linear combination of forward activation maps followed by a ReLU non-linearity yields the localization heatmap:

$$L_{\text{Grad-CAM}}^c = \text{ReLU}\left(\sum_k \alpha_k^c A^k \right) \tag{1.17}$$

   The resulting heatmap is normalized by dividing by its maximum value and bilinearly upsampled to the input image resolution for visual inspection.

Chapter 2

Experimental Setup and Reproducibility

To guarantee strict reproducibility across all experiments, all random seeds, data split sizes, hardware details, and execution parameters were standardized. Every reported result represents a single deterministic run with seed 42. Given that the validation set contains 50 images and the test set contains 100 images in Part B, differences within ±8–10 percentage points fall within expected statistical sampling variance.

2.1 Datasets and Data Partitions

Two distinct visual benchmarks are investigated:

2.1.1 Part A: Five-Class MNIST Subset

The MNIST dataset [13] contains 28×28 grayscale images of handwritten digits. For Part A, a five-class subset is isolated:

$$\mathcal{C}_{\text{MNIST}} = \{3, 4, 5, 7, 8\} \tag{2.1}$$

Images are normalized such that pixel intensities lie in the range $[0.0, 1.0]$. The subset is partitioned into standard training, validation, and testing splits matching the 60/20/20 ratio, detailed in Table 2.1.

Table 2.1: MNIST Five-Class Subset Split Statistics (Part A).

Digit Class	Train Set	Validation Set	Test Set	Total Images
Digit 3	2,277	759	759	3,795
Digit 4	2,277	759	759	3,795
Digit 5	2,277	759	759	3,795
Digit 7	2,277	759	759	3,795
Digit 8	2,277	759	759	3,795
Total	**11,385**	**3,795**	**3,795**	**18,975**

2.1.2 Part B: Five-Class Caltech-101 Visual Subset

Caltech-101 [12] consists of natural RGB photographs across 101 object categories. For Part B, a five-category subset is curated:

$$\mathcal{C}_{\text{Caltech}} = \{\text{butterfly}, \text{car_side}, \text{grand_piano}, \text{scorpion}, \text{starfish}\} \tag{2.2}$$

Images are resized to 224×224 RGB. For custom CNNs, pixel intensities are scaled to $[0.0, 1.0]$ without channel mean/standard deviation normalization and without data augmentation. For VGG-19, inputs are preprocessed using the standard Keras VGG-19 preprocessing pipeline, which performs RGB-to-BGR channel conversion followed by ImageNet channel-mean subtraction ($[103.939, 116.779, 123.68]$). Table 2.2 details the split statistics.

Table 2.2: Caltech-101 Five-Category Dataset Split Statistics (Part B).

Category	Train Set	Validation Set	Test Set	Total Images
`butterfly`	50	10	20	80
`car_side`	50	10	20	80
`grand_piano`	50	10	20	80
`scorpion`	50	10	20	80
`starfish`	50	10	20	80
Total	**250**	**50**	**100**	**400**

Chapter 3

Part A: Feature Extraction and Classification on MNIST

Part A systematically deconstructs the convolutional pipeline on the five-class MNIST dataset ($\{3, 4, 5, 7, 8\}$), progressing from isolated single-kernel spatial filtering to stacked multi-layer representation learning, culminating in a trained end-to-end classifier with receptive field attribution.

3.1 Part A Q1: Isolated Single Filter Convolution Analysis

3.1.1 Kernel Specification and Spatial Dimension Verification

To investigate the fundamental behavior of an untrained spatial filter, a single 5×5 convolutional kernel ($K = 5$) was initialized using Kaiming normal initialization with `mode="fan_in"` and ReLU gain (std $= \sqrt{2/25} \approx 0.283$, with bias initialized to zero). The exact 5×5 weight matrix obtained under seed 42 is tabulated in Table 3.1.

Table 3.1: Single 5×5 Kaiming-Initialized Kernel Weights (Seed 42).

$w_{0,0}$	$w_{0,1}$	$w_{0,2}$	$w_{0,3}$	$w_{0,4}$
0.0549	0.6113	−0.0487	0.2402	−0.5443
0.1847	−0.1837	−0.2312	0.1493	−0.3607
−0.4701	−0.0858	−0.0262	0.0564	−0.3169
0.5254	−0.2021	0.1946	0.2254	−0.0094
0.4219	−0.1461	−0.0719	0.4171	−0.0922

The numerical entries of this kernel sum to $\approx +0.291$, confirming that it is not a zero-sum

kernel. For an input image of dimensions $W_{in} = H_{in} = 28$, stride $S = 1$, and zero padding $P = 0$, the theoretical output spatial dimension is computed via Eq. (1.4):

$$W_{out} = H_{out} = \left\lfloor \frac{28 - 5 + 2(0)}{1} \right\rfloor + 1 = 23 + 1 = 24 \tag{3.1}$$

Both the expected spatial size (24×24) and the obtained spatial size (24×24) match exactly.

3.1.2 Manual Sliding-Window vs. Framework Implementation

To verify the algorithmic correctness of Eq. (1.3), a manual 2D sliding-window cross-correlation was implemented using nested spatial loops:

$$Y_{\text{manual}}[i, j] = \sum_{m=0}^{4} \sum_{n=0}^{4} X[i + m, j + n] \cdot W[m, n] + b \tag{3.2}$$

The output was numerically compared against `torch.nn.Conv2d(1, 1, kernel_size=5, bias=True)`. Across all spatial coordinates and all five representative test samples, the maximum absolute difference was:

$$\max_{i,j} |Y_{\text{manual}}[i, j] - Y_{\text{Conv2d}}[i, j]| = 3.43 \times 10^{-7} \tag{3.3}$$

which is orders of magnitude below the standard 10^{-5} floating-point tolerance, confirming algorithmic equivalence.

3.1.3 Feature Map Visualizations Across All Digits

Figure 3.1 displays the single 5×5 kernel weights alongside the input digits and their corresponding output feature maps for all five representative MNIST digits $(3, 4, 5, 7, 8)$.

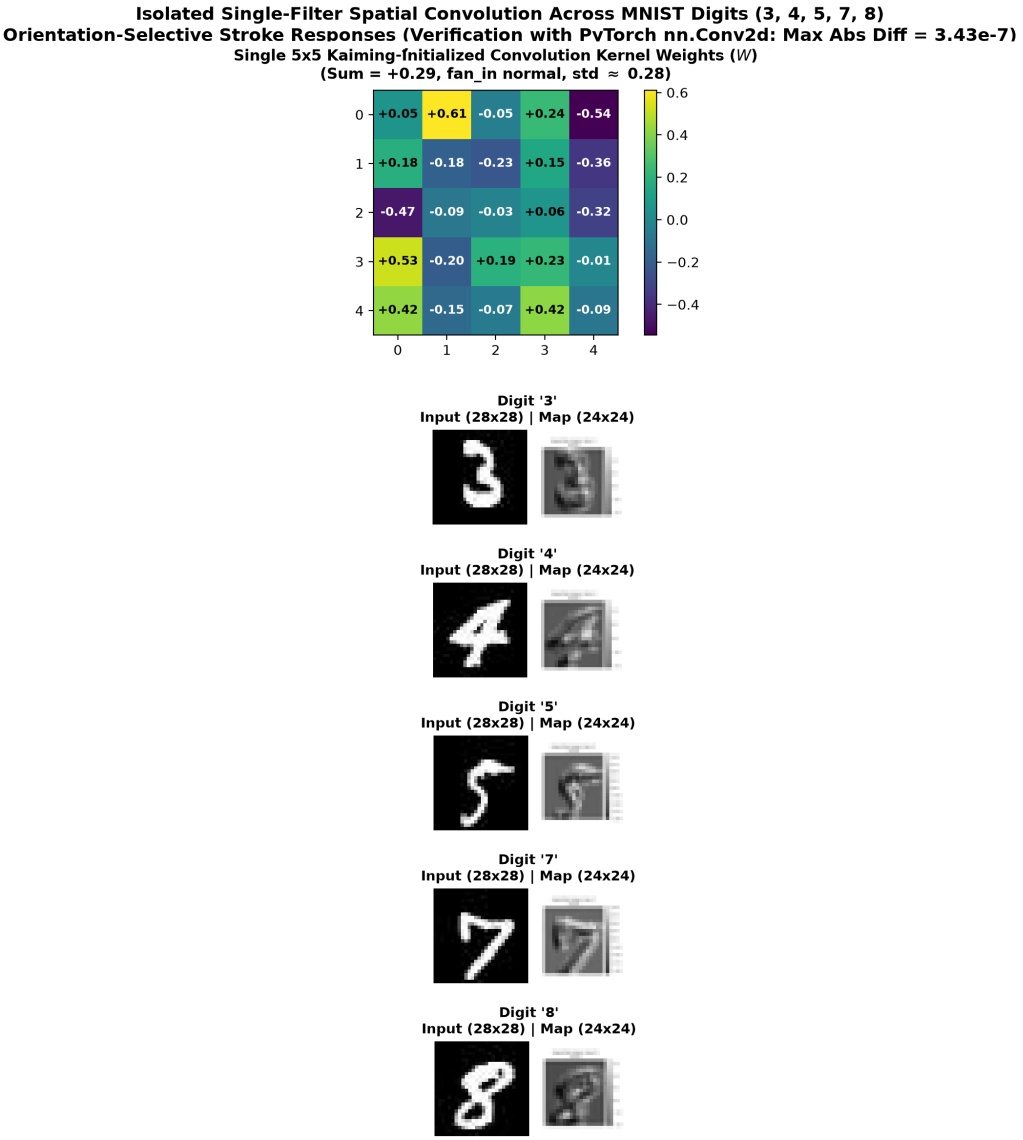


Figure 3.1: Single 5×5 convolution filter analysis: centered kernel weights followed by input images and corresponding feature maps for digits 3, 4, 5, 7, and 8, stacked vertically.

3.1.4 Observation

Observation: Positive weights (e.g., $+0.6113$ in the top row, and $+0.5254, +0.4219$ in the lower-left corner) produce high activations where pixel strokes align with these coefficients, whereas negative weights (e.g., -0.5443 and -0.4701) suppress non-matching stroke boundaries. Consequently, the output operates as an oriented, stroke-sensitive response. Even without optimization, random spatial filtering extracts basic directional stroke cues, although it lacks class-discriminative structure.

3.2 Part A Q2: Two-Stage Hierarchical Feature Extraction

3.2.1 Architectural Hierarchy and Shape Verification

To investigate hierarchical spatial contraction and non-linear feature transformation, a two-stage sequential block was constructed:

$$\text{Input } (1\times28\times28) \xrightarrow{\text{Conv1}} 6\times24\times24 \xrightarrow{\text{ReLU}} \xrightarrow{\text{Pool1}} 6\times12\times12 \xrightarrow{\text{Conv2}} 16\times8\times8 \xrightarrow{\text{ReLU}} \xrightarrow{\text{Pool2}} 16\times4\times4 \tag{3.4}$$

The layer-by-layer spatial transformations, governing formulas, expected shapes, and experimentally verified shapes are documented in Table 3.2.

Table 3.2: Layer-by-Layer Shape Verification in Two-Stage Stacked Feature Extractor.

Stage / Layer	**Governing Formula**	**Expected Shape**	**Obtained Shape**	**Match**
Input	Initial image dimensions	$(5, 1, 28, 28)$	$(5, 1, 28, 28)$	Yes
Conv1 (5×5, $S=1, P=0$)	$\lfloor(28-5+0)/1\rfloor+1=24$	$(5, 6, 24, 24)$	$(5, 6, 24, 24)$	Yes
ReLU1	$\max(0, x)$	$(5, 6, 24, 24)$	$(5, 6, 24, 24)$	Yes
Pool1 (Max, 2×2, $S=2$)	$\lfloor(24-2)/2\rfloor+1=12$	$(5, 6, 12, 12)$	$(5, 6, 12, 12)$	Yes
Conv2 (5×5, $S=1, P=0$)	$\lfloor(12-5+0)/1\rfloor+1=8$	$(5, 16, 8, 8)$	$(5, 16, 8, 8)$	Yes
ReLU2	$\max(0, x)$	$(5, 16, 8, 8)$	$(5, 16, 8, 8)$	Yes
Pool2 (Max, 2×2, $S=2$)	$\lfloor(8-2)/2\rfloor+1=4$	$(5, 16, 4, 4)$	$(5, 16, 4, 4)$	Yes

3.2.2 Filter Visualizations

Figure 3.2 illustrates three selected filters from Conv1 (indices 0, 1, 2) and three from Conv2 (indices 0, 1, 2). Because Conv2 filters process 6 input channels (dimensions $6\times5\times5$), each Conv2 filter is visualized by computing the mean across all 6 input channels:

$$\bar{K}_{\text{conv2}}[m, n] = \frac{1}{6}\sum_{c=1}^{6} K_{\text{conv2}}[c, m, n] \tag{3.5}$$

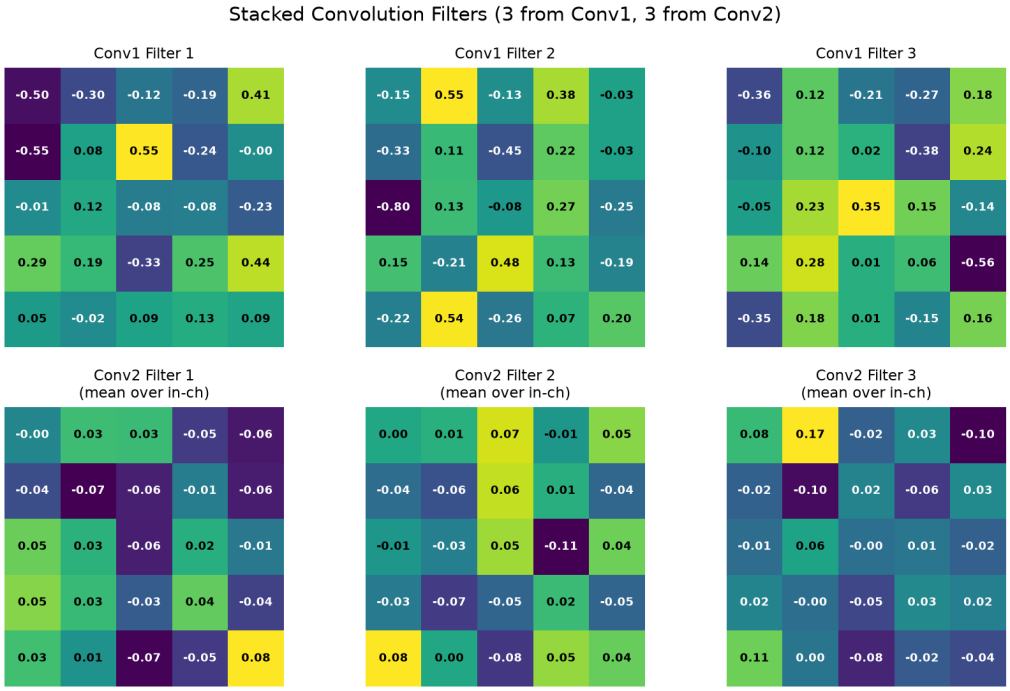
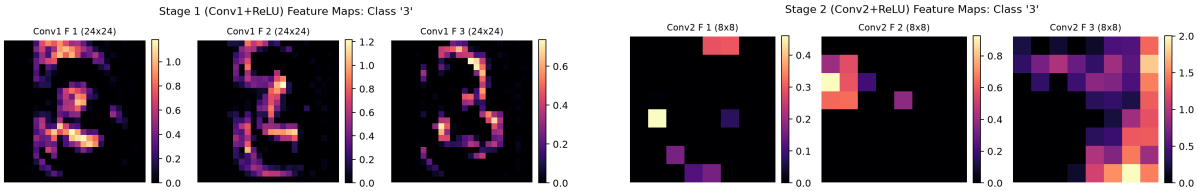


Figure 3.2: Selected untrained filters from Conv1 ($1 \times 5 \times 5$) and Conv2 ($6 \times 5 \times 5$, displayed as channel-mean).

3.2.3 Feature Map Progression and Tensor Logging

Feature maps after each transformation were extracted and logged to the project archive. Figure 3.3 depicts the resulting post-ReLU feature maps (24×24 and 8×8) for representative digit 3.



(a) Stage 1: Conv1 + ReLU (24×24)

(b) Stage 2: Conv2 + ReLU (8×8)

Figure 3.3: Feature maps across Stage 1 and Stage 2 for representative digit 3 (showing post-ReLU activations before pooling).

Inspection of the logged tensor slices (logged for channel 0 across all five representative images) highlights the numerical impact of non-linear thresholding and pooling:

- After Conv1 (24×24), activations span both positive and negative values ($Y_{\text{conv1}}[0,0,0,6] = +0.7113$, while $Y_{\text{conv1}}[0,0,3,10] = -0.4277$).
- After ReLU1, negative activations are thresholded to zero. Across all five images and six channels, **50.9**% of post-ReLU activations are exactly zero (52.3% for the digit 3 image).

- After Pool1 (12×12), only local maxima survive ($Y_{\text{pool1}}[0, 0, 0, 3] = 0.8743$).
- At Conv2 (8×8) and Pool2 (4×4), activations become condensed and sparse.

3.2.4 Observation

Observation: Spatial resolution shrinks systematically through the hierarchy ($24 \rightarrow 12 \rightarrow 8 \rightarrow 4$) while each individual feature unit integrates information across an increasingly broad region of the digit. Conv1 feature maps visually capture stroke boundaries and gradients, whereas Conv2 feature maps are coarse and sparse due to repeated non-linear thresholding. Because these filters are randomly initialized, activations provide unselective responses that fire uniformly across strokes.

3.3 Part A Q3a: Supervised Classification on MNIST Subset

3.3.1 Model Architecture and Hyperparameter Configuration

To evaluate learned representations, an end-to-end classifier was constructed incorporating two sequential `ConvReLUPool` blocks followed by a fully connected classification head:

- `ConvReLUPool 1`: Conv2d(1, 6, $K = 5$, $S = 1$, $P = 0$), ReLU, MaxPool2d(2×2, $S = 2$). Output: $6 \times 12 \times 12$.
- `ConvReLUPool 2`: Conv2d(6, 16, $K = 5$, $S = 1$, $P = 0$), ReLU, MaxPool2d(2×2, $S = 2$). Output: $16 \times 4 \times 4$.
- `Classifier Head`: Flatten ($16 \times 4 \times 4 = 256$) $\rightarrow$ Linear(256, 64) $\rightarrow$ ReLU $\rightarrow$ Linear(64, 5).

Convolutional layers were initialized with Kaiming normal (`mode="fan_in"`, ReLU gain, zero biases), while linear layers utilized Xavier uniform initialization (bound $\pm\sqrt{6/(n_{\text{in}} + n_{\text{out}})}$, zero biases). The network was trained using mini-batch SGD with learning rate $\eta = 0.02$, momentum $\mu = 0.0$, batch size 64, for 15 epochs. Softmax normalization was applied strictly at evaluation. Model checkpoints were selected based on peak validation accuracy.

3.3.2 Quantitative Results and Training Dynamics

The classification performance across data partitions is presented in Table 3.3.

Table 3.3: Classification Performance of Trained Digit Classifier on MNIST Subset.

Data Partition	**Loss**[1]	**Accuracy (%)**	**Total Samples**
Training Set	0.0351	98.89%	11,385
Validation Set	0.0504	98.47%	3,795
Test Set	0.0631	**98.02%**	3,795

Figure 3.4 illustrates the loss and accuracy trajectories over 15 training epochs, while Figure 3.5 provides the confusion matrices for training, validation, and testing sets.

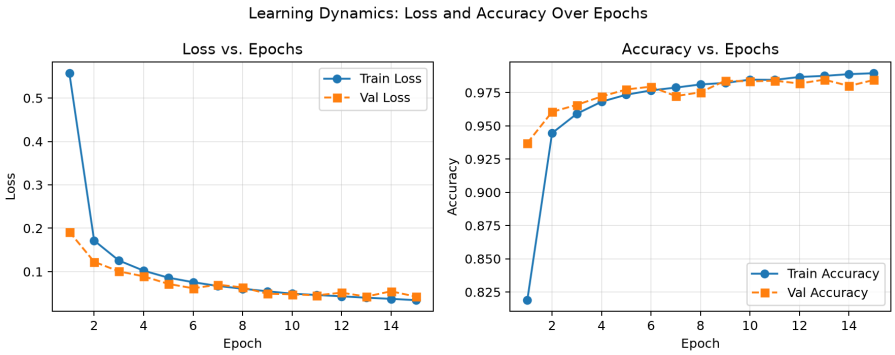


Figure 3.4: Training and validation loss and accuracy curves for digit classifier over 15 epochs.

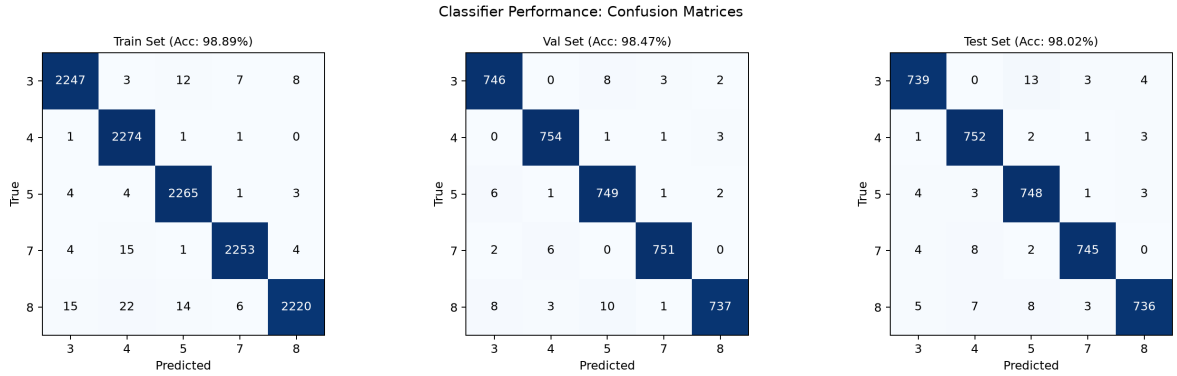


Figure 3.5: Confusion matrices on MNIST training, validation, and test splits (Classes: 3, 4, 5, 7, 8).

3.3.3 Observation

Observation: The generalization gap between training accuracy (98.89%) and test accuracy (98.02%) is 0.87 percentage points ($\approx$ 0.9 pp), indicating well-regularized convergence. Training

[1]Loss values reflect Cross-Entropy loss recorded during model training and evaluation.

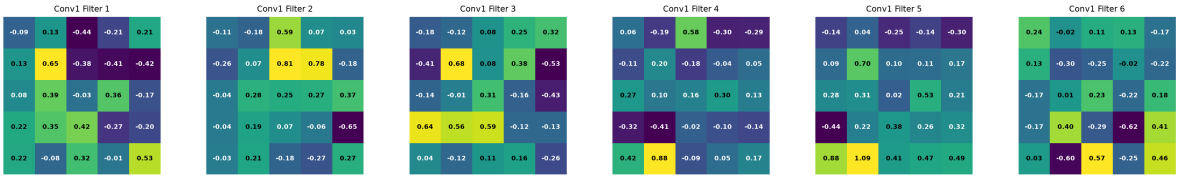
and validation losses decrease together; validation loss exhibits minor fluctuations after epoch ≈ 10.

Examining class-level errors on the test set, digit 8 exhibits the lowest recall (736/759 = 96.97%), followed by digit 3 (739/759 = 97.36%). The remaining digits achieve higher recall: digit 7 (98.16%), digit 5 (98.55%), and digit 4 (99.08%). The largest single off-diagonal test confusion occurs between digits 3 and 5 ($3 \rightarrow 5$: 13 misclassifications). In the training set, the most frequent confusions are $8 \rightarrow 4$ (22 misclassifications), $7 \rightarrow 4$ (15 misclassifications), $8 \rightarrow 3$ (15 misclassifications), $8 \rightarrow 5$ (14 misclassifications), and $3 \rightarrow 5$ (12 misclassifications). A plausible morphological hypothesis for these patterns is that digits 3, 5, and 8 share curved lower loops, whereas digits 4 and 7 share prominent diagonal stems.

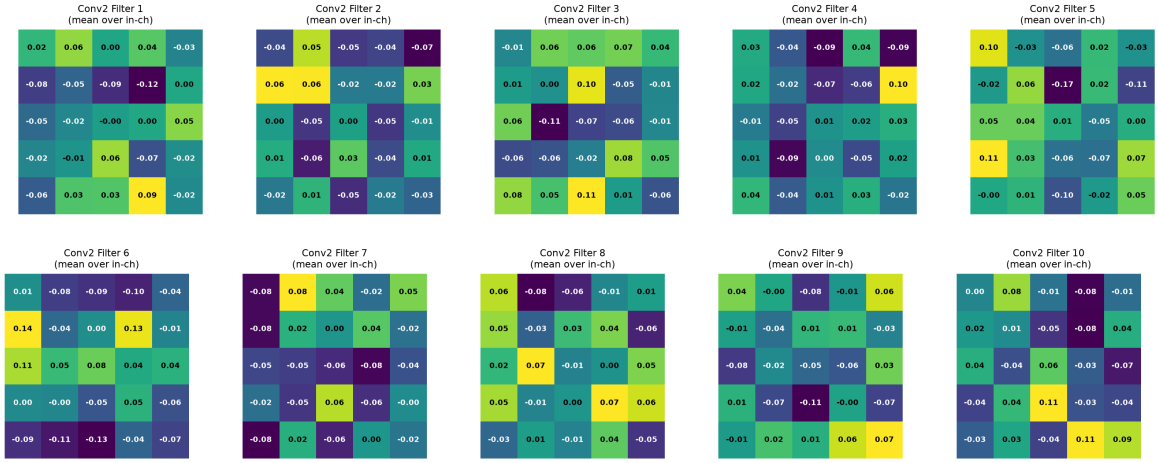
3.4 Part A Q3b: Learned Convolutional Filters and Feature Representations

3.4.1 Filter Architecture Clarification

Note on Filter Counts: The requested configuration specifies 10 Conv1 filters; however, our Conv1 layer comprises 6 filters, matching the standard LeNet design. Therefore, Figure 3.6 displays all 6 learned Conv1 filters and the first 10 of 16 learned Conv2 filters from the best-validation checkpoint (visualized as the mean over all 6 input channels) in separate, enlarged panels.

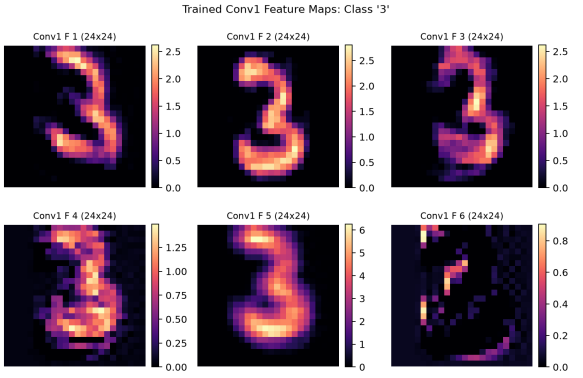


(a) Conv1: all six learned filters.

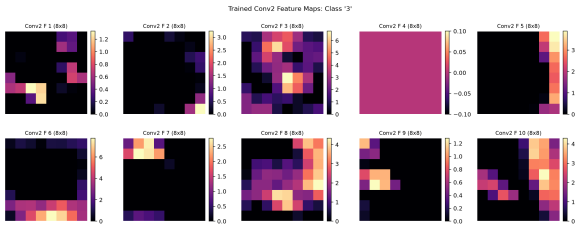


(b) Conv2: the first ten filters, averaged over input channels.

Figure 3.6: Learned convolutional filters from the best-validation checkpoint, shown in separate enlarged panels.



(a) Digit 3 Conv1 Feature Maps



(b) Digit 3 Conv2 Feature Maps

Figure 3.7: Feature map representations of representative digit 3 generated by the trained network.

3.4.2 Observation

Observation: Unlike the isotropic random filters in Q2, several learned Conv1 filters exhibit clear row and column spatial structures (e.g., horizontal stroke detectors, vertical edge filters, and diagonal gradients), while others retain higher-frequency noise. Conv2 weights exhibit lower

variance and distributed spatial patterns. The resulting feature maps transition from outline responses in Conv1 to sparse, localized activations in Conv2. In Figure 3.7b, filter 4 in Conv2 remains inactive for this digit-3 input, demonstrating channel selectivity.

3.5 Part A Q3c: Maximum Activation Neurons and Receptive Field Mapping

3.5.1 Theoretical Receptive Field Derivation

For the digit classifier, the receptive field of neurons at the output of Conv2 (8×8) before pooling is computed step-by-step using Eq. (1.7)–(1.9):

1. **Input Image:** $RF_0 = 1,\ J_0 = 1, start_0 = 0.5$.
2. **Conv1 Layer ($K = 5, S = 1, P = 0$):**

$$RF_{\text{conv1}} = 1 + (5-1) \cdot 1 = \mathbf{5}, \quad J_{\text{conv1}} = 1 \cdot 1 = 1, \quad start_{\text{conv1}} = 0.5 + 2 = 2.5 \tag{3.6}$$

3. **Pool1 Layer ($K = 2, S = 2$):**

$$RF_{\text{pool1}} = 5 + (2-1) \cdot 1 = \mathbf{6}, \quad J_{\text{pool1}} = 1 \cdot 2 = 2, \quad start_{\text{pool1}} = 2.5 + 0.5 = 3.0 \tag{3.7}$$

4. **Conv2 Layer ($K = 5, S = 1, P = 0$):**

$$RF_{\text{conv2}} = 6 + (5-1) \cdot 2 = \mathbf{14}, \quad J_{\text{conv2}} = 2 \cdot 1 = 2, \quad start_{\text{conv2}} = 3.0 + 4 = 7.0 \tag{3.8}$$

Thus, every activation unit in the $16 \times 8 \times 8$ feature map of Conv2 has an exact theoretical receptive field of $\mathbf{14} \times \mathbf{14}$ pixels in the 28×28 input image, with spatial jump $J = 2$ and start coordinate 7.0.

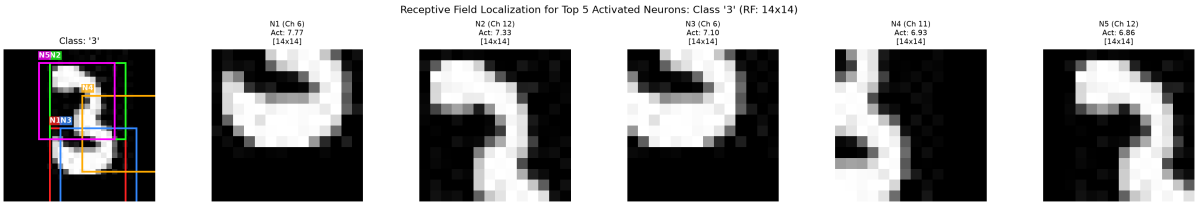
3.5.2 Top-5 Activating Neurons and Channel Indexing

Channel Indexing Convention: All channels are indexed from 0, consistent with the internal implementations. Top-5 neurons are selected across the entire $16 \times 8 \times 8$ feature map volume per image.

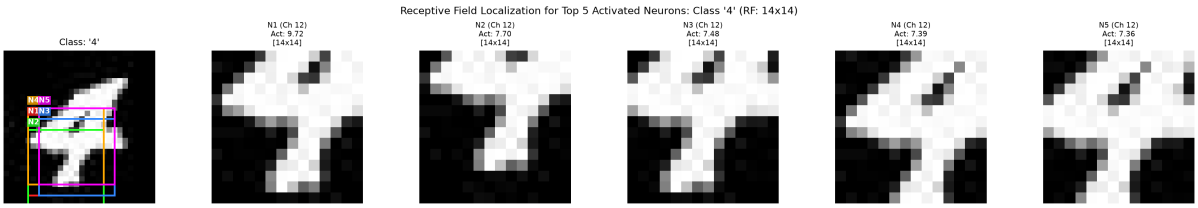
Figure 3.8 illustrates the corresponding 14×14 receptive field bounding boxes mapped onto the original input images, with each digit shown in a separate, enlarged row for clear inspection.

Table 3.4: Top-5 Activating Neurons at Conv2 Output Across Five Representative MNIST Digits.

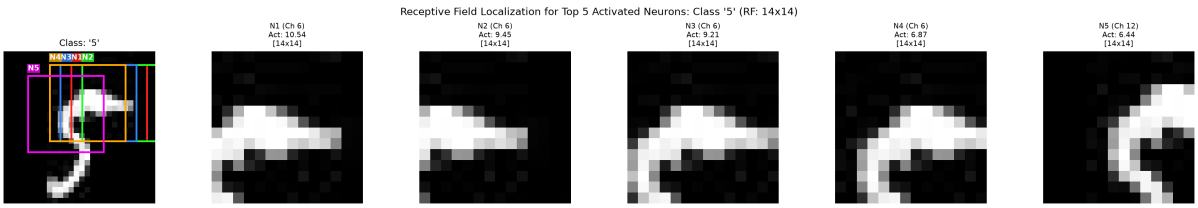
Digit	Rank	Ch (0-idx)	Spatial (Row, Col)	Activation	Input BBox $[y_1, x_1, y_2, x_2]$
	1	Ch 5	$(7, 4)$	7.767	$[14, 8, 28, 22]$
	2	Ch 11	$(1, 4)$	7.331	$[2, 8, 16, 22]$
3	3	Ch 5	$(7, 5)$	7.098	$[14, 10, 28, 24]$
	4	Ch 10	$(4, 7)$	6.932	$[8, 14, 22, 28]$
	5	Ch 11	$(1, 3)$	6.859	$[2, 6, 16, 20]$
	1	Ch 11	$(6, 2)$	9.717	$[12, 4, 26, 18]$
	2	Ch 11	$(7, 2)$	7.703	$[14, 4, 28, 18]$
4	3	Ch 11	$(6, 3)$	7.485	$[12, 6, 26, 20]$
	4	Ch 11	$(5, 2)$	7.394	$[10, 4, 24, 18]$
	5	Ch 11	$(5, 3)$	7.361	$[10, 6, 24, 20]$
	1	Ch 5	$(1, 6)$	10.540	$[2, 12, 16, 26]$
	2	Ch 5	$(1, 7)$	9.447	$[2, 14, 16, 28]$
5	3	Ch 5	$(1, 5)$	9.209	$[2, 10, 16, 24]$
	4	Ch 5	$(1, 4)$	6.875	$[2, 8, 16, 22]$
	5	Ch 11	$(2, 2)$	6.444	$[4, 4, 18, 18]$
	1	Ch 2	$(3, 5)$	8.463	$[6, 10, 20, 24]$
	2	Ch 11	$(3, 6)$	7.524	$[6, 12, 20, 26]$
7	3	Ch 11	$(4, 5)$	7.458	$[8, 10, 22, 24]$
	4	Ch 5	$(1, 4)$	7.318	$[2, 8, 16, 22]$
	5	Ch 15	$(6, 5)$	7.316	$[12, 10, 26, 24]$
	1	Ch 2	$(2, 5)$	7.639	$[4, 10, 18, 24]$
	2	Ch 11	$(3, 1)$	7.062	$[6, 2, 20, 16]$
8	3	Ch 11	$(4, 2)$	6.969	$[8, 4, 22, 18]$
	4	Ch 11	$(4, 1)$	6.690	$[8, 2, 22, 16]$
	5	Ch 2	$(2, 4)$	6.644	$[4, 8, 18, 22]$



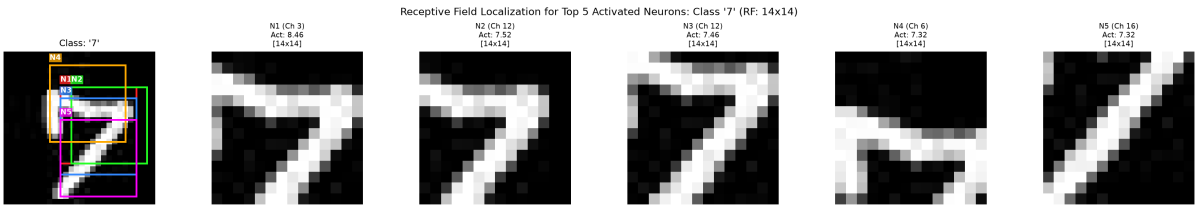
(a) Digit 3



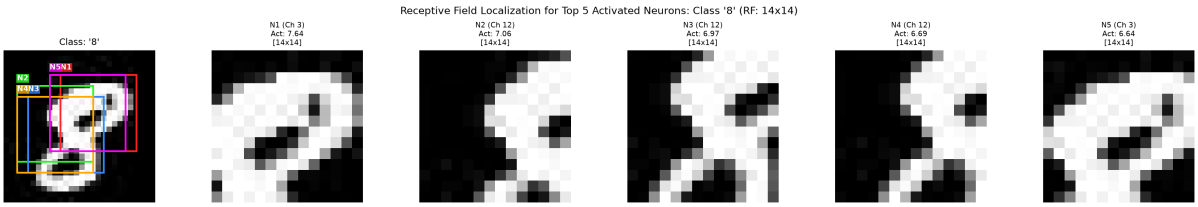
(b) Digit 4



(c) Digit 5



(d) Digit 7



(e) Digit 8

Figure 3.8: Receptive field bounding boxes (14×14 pixels) for top-5 activating neurons overlaid on input digits.

3.5.3 Observation

Observation: For these representative samples, only digit 4 is dominated by a single channel: all five top activations originate from channel 11 (1-indexed 12) along its vertical stem and fork. Digit 5 utilizes channels 5 and 11 (4 activations from channel 5 and 1 from channel 11). Digit 8 utilizes channels 2 and 11. Digit 3 activates channels 5, 11, and 10. Digit 7 activates channels

2, 11, 5, and 15. While these specific channel alignments suggest localized stroke specialization on the evaluated samples, confirming class-wide stroke specialization would require aggregating across larger sets of test instances.

Chapter 4

Part B (Q1): Custom CNN Architectures and Dropout Study on Caltech-101

Part B investigates convolutional architectures on natural images from Caltech-101 under extreme data scarcity (250 training images across five categories: *butterfly*, *car_side*, *grand_piano*, *scorpion*, and *starfish*). This chapter carries out an empirical study spanning three architectural depths (4-layer, 5-layer, and 6-layer) under dropout rates $p \in \{0.0, 0.3, 0.5\}$.

4.1 Part B Q1a: Custom CNN Architectural Exploration

4.1.1 Architectural Configurations and Dimension Verification

All custom CNNs operate with zero padding ($P = 0$) on all convolutional layers. Max-pooling layers use kernel size 3×3 with stride 2. Each architecture terminates in an identical fully connected classification head: `Flatten` $\rightarrow$ `Linear(dim, 128)` $\rightarrow$ `ReLU` $\rightarrow$ `Dropout(p)` $\rightarrow$ `Linear(128, 5)`.

1. **Architecture 1 (4-Layer: 2 Conv + 2 FC):**
   - Conv1: $3 \rightarrow 8$, $K = 11, S = 4, P = 0 \implies \lfloor (224 - 11)/4 \rfloor + 1 = 54 \times 54$.
   - Pool1: Max-Pool $3 \times 3, S = 2 \implies \lfloor (54 - 3)/2 \rfloor + 1 = 26 \times 26$.
   - Conv2: $8 \rightarrow 16$, $K = 5, S = 1, P = 0 \implies \lfloor (26 - 5)/1 \rfloor + 1 = 22 \times 22$.
   - Pool2: Max-Pool $3 \times 3, S = 2 \implies \lfloor (22 - 3)/2 \rfloor + 1 = 10 \times 10$.
   - Flatten size: $16 \times 10 \times 10 = \mathbf{1,600}$.
   - Parameters: Conv1: 2,912; Conv2: 3,216; FC1: 204,928; FC2: 645. Total: $\mathbf{211,701}$.

2. **Architecture 2 (5-Layer: 3 Conv + 2 FC):**

   - Conv1 through Pool2 identical to Arch 1 ($16 \times 10 \times 10$).
   - Conv3: $16 \to 32$, $K = 3, S = 1, P = 0 \implies \lfloor (10-3)/1 \rfloor + 1 = 8 \times 8$.
   - Pool3: Max-Pool $3 \times 3, S = 2 \implies \lfloor (8-3)/2 \rfloor + 1 = 3 \times 3$.
   - Flatten size: $32 \times 3 \times 3 = \mathbf{288}$.
   - Parameters: Conv1: 2,912; Conv2: 3,216; Conv3: 4,640; FC1: 36,992; FC2: 645. Total: $\mathbf{48,405}$.

3. **Architecture 3 (6-Layer: 4 Conv + 2 FC):**

   - Conv1 through Conv2 and Pool2 identical to Arch 1 ($16 \times 10 \times 10$).
   - Conv3: $16 \to 32$, $K = 3, S = 1, P = 0 \implies 8 \times 8$ (**no pooling applied after Conv3**).
   - Conv4: $32 \to 64$, $K = 3, S = 1, P = 0 \implies \lfloor (8-3)/1 \rfloor + 1 = 6 \times 6$.
   - Pool4: Max-Pool $3 \times 3, S = 2 \implies \lfloor (6-3)/2 \rfloor + 1 = 2 \times 2$.
   - Flatten size: $64 \times 2 \times 2 = \mathbf{256}$.
   - Parameters: Conv1: 2,912; Conv2: 3,216; Conv3: 4,640; Conv4: 18,496; FC1: 32,896; FC2: 645. Total: $\mathbf{62,805}$.

Table 4.1 documents the layer-by-layer output dimensions across all three architectures. Table 4.2 summarizes parameter allocation. In Architecture 1, FC1 accounts for 96.8% of all parameters.

Table 4.1: Layer-by-Layer Feature Map Shapes Across Custom CNN Architectures.

Stage	**Layer Specification**	**Arch 1 (4-Layer)**	**Arch 2 (5-Layer)**	**Arch 3 (6-Layer)**
Input	Initial Caltech Image	$3 \times 224 \times 224$	$3 \times 224 \times 224$	$3 \times 224 \times 224$
Conv1	Conv2d($3 \to 8, K = 11, S = 4, P = 0$)	$8 \times 54 \times 54$	$8 \times 54 \times 54$	$8 \times 54 \times 54$
Pool1	MaxPool2d($K = 3, S = 2$)	$8 \times 26 \times 26$	$8 \times 26 \times 26$	$8 \times 26 \times 26$
Conv2	Conv2d($8 \to 16, K = 5, S = 1, P = 0$)	$16 \times 22 \times 22$	$16 \times 22 \times 22$	$16 \times 22 \times 22$
Pool2	MaxPool2d($K = 3, S = 2$)	$16 \times 10 \times 10$	$16 \times 10 \times 10$	$16 \times 10 \times 10$
Conv3	Conv2d($16 \to 32, K = 3, S = 1, P = 0$)	—	$32 \times 8 \times 8$	$32 \times 8 \times 8$
Pool3	MaxPool2d($K = 3, S = 2$)	—	$32 \times 3 \times 3$	—
Conv4	Conv2d($32 \to 64, K = 3, S = 1, P = 0$)	—	—	$64 \times 6 \times 6$
Pool4	MaxPool2d($K = 3, S = 2$)	—	—	$64 \times 2 \times 2$
Flatten	Linear Vectorization	1,600	288	256
FC1	Linear(dim, 128) + ReLU + Dropout	128	128	128
FC2 (Out)	Linear(128, 5)	5	5	5
Total Parameters		**211,701**	**48,405**	**62,805**

Table 4.2: Architectural Comparison and Parameter Distribution Across Custom CNNs.

Architecture	Conv Layers	Flatten Size	FC1 Parameters	Total Parameters
Architecture 1	2	1,600	204,928 (96.8%)	211,701
Architecture 2	3	288	36,992 (76.4%)	48,405
Architecture 3	4	256	32,896 (52.4%)	62,805

4.1.2 Training Protocol and Optimization

All nine models were trained using the Adam optimizer [8] with learning rate $\eta = 10^{-3}$, weight decay 10^{-4}, batch size 16, for up to 100 epochs, without data augmentation. Convolution weights were initialized with Kaiming normal (`mode="fan_out"`), and linear layers with Xavier uniform. Training incorporated early stopping with a patience of 15 epochs on validation accuracy (with validation loss as tie-breaker, min delta 10^{-4}) and an early termination check if training loss dropped below 10^{-4}. Checkpoints within a run were saved based on validation accuracy and validation loss. The across-model selection key followed:

$$\text{selection_key} = (\text{val_acc}, -\text{val_loss}, -\text{param_count}) \tag{4.1}$$

Test accuracy was strictly excluded from all model selection decisions.

4.2 Part B Q1b: Hyperparameter Search and Dropout Regularization Study

4.2.1 Comprehensive Results Across All Nine Configurations

Table 4.3 presents the complete empirical metrics across all nine configurations spanning dropout rates $p \in \{0.0, 0.3, 0.5\}$.

Figure 4.1 illustrates the comparative grouped bar chart directly contrasting training, validation, and test accuracies across all nine runs. Figure 4.2 displays the training loss and validation accuracy curves for the three architectures under $p = 0.3$. Figure 4.3 displays the complete matrix of train, validation, and test confusion matrices for all three architectures at $p = 0.3$.

Table 4.3: Performance Metrics Across All 9 Custom CNN Models on Caltech-101 Subset.

Dropout (p)	Arch	Train (%)	Val (%)	Test (%)	Val Loss	Best Ep	Run Ep	Parameters
	Arch 1	100.0%	86.0%	74.0%	0.6575	21	36	211,701
0.0	Arch 2	94.8%	78.0%	80.0%	0.6828	17	32	48,405
	Arch 3	98.4%	74.0%	62.0%	1.0187	31	46	62,805
	Arch 1	**98.8%**	**90.0%**	**73.0%**	**0.4849**	**18**	**33**	**211,701**
0.3	Arch 2	98.4%	80.0%	83.0%	0.8840	32	47	48,405
	Arch 3	91.6%	78.0%	70.0%	0.6468	20	35	62,805
	Arch 1	99.6%	86.0%	76.0%	0.5471	34	49	211,701
0.5	Arch 2	97.6%	86.0%	81.0%	0.5864	26	41	48,405
	Arch 3	93.2%	84.0%	78.0%	0.6135	24	39	62,805

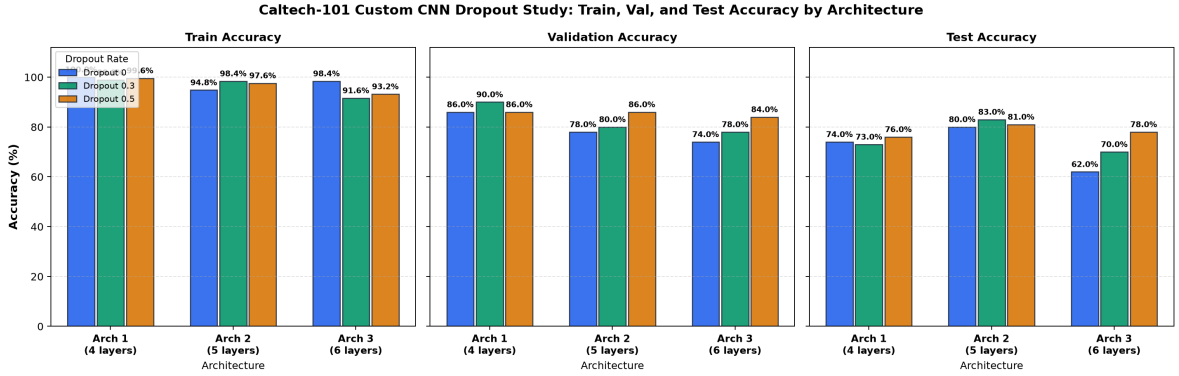
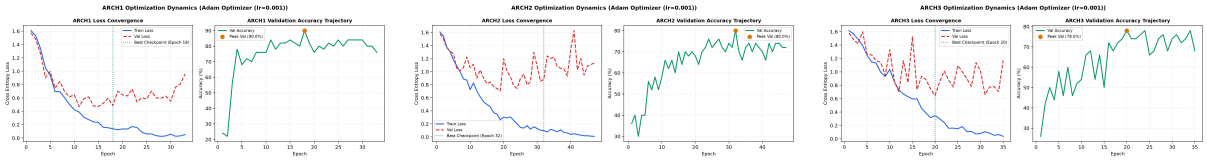


Figure 4.1: Comparative grouped bar chart of training, validation, and test accuracies for all 9 custom CNN models.



(a) Arch 1 ($p = 0.3$) (b) Arch 2 ($p = 0.3$) (c) Arch 3 ($p = 0.3$)

Figure 4.2: Training loss and validation accuracy curves for custom CNN architectures under dropout $p = 0.3$.

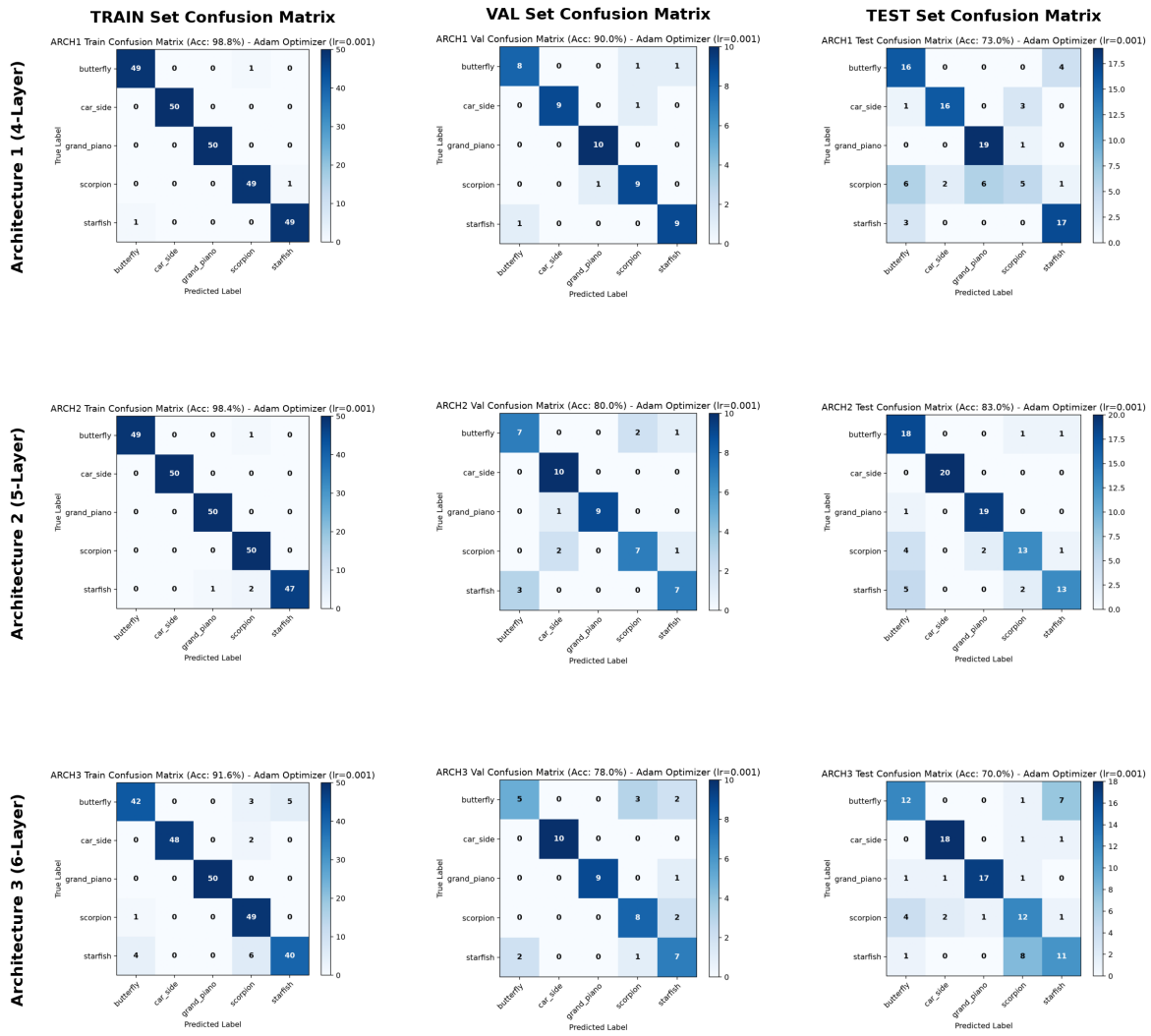


Figure 4.3: Complete confusion matrix evaluation for custom CNN architectures under dropout $p = 0.3$ across train, validation, and test splits.

4.2.2 Observations on Regularization, Depth, and Error Distributions

Observations:

1. **Overfitting Under Small Sample Regime:** Every configuration exhibits substantial overfitting: training accuracy reaches $\geq 91.6\%$ across all runs (100.0% in Arch 1 at $p = 0.0$), while validation accuracy is bounded between 74.0% and 90.0%. In unregularized Arch 1 ($p = 0.0$), validation loss reaches a minimum around epoch 21 (0.6575) and subsequently deteriorates while training loss continues to descend.

2. **Generalization Gap Trends:** On average, dropout contracts the generalization gap

(training accuracy minus validation accuracy), moving from 18.4 percentage points ($p = 0.0$) $\rightarrow$ 13.6 percentage points ($p = 0.3$) $\rightarrow$ 11.5 percentage points ($p = 0.5$). However, this reduction is **not strictly monotonic per architecture**: for Arch 1, the gaps are 14.0 $\rightarrow$ 8.8 $\rightarrow$ 13.6 pp; for Arch 2, 16.8 $\rightarrow$ 18.4 $\rightarrow$ 11.6 pp; and for Arch 3, 24.4 $\rightarrow$ 13.6 $\rightarrow$ 9.2 pp.

3. **Depth and Dropout Interaction:** The deepest model (Arch 3) exhibits the largest improvement in test accuracy with dropout, rising from 62.0% ($p = 0.0$) to 70.0% ($p = 0.3$) and 78.0% ($p = 0.5$). However, Arch 3 consistently achieves lower test accuracy than the more compact Arch 2 at $p = 0.0$ and $p = 0.3$. At $p = 0.5$, Arch 3 ranks second in test accuracy (78.0%), ahead of Arch 1 (76.0%), but still below Arch 2 (81.0%).

4. **Convergence Dynamics and Trajectory Noise:** The epoch of best validation performance does not shift monotonically with dropout: for Arch 1, it follows 21 $\rightarrow$ 18 $\rightarrow$ 34; for Arch 2, 17 $\rightarrow$ 32 $\rightarrow$ 26; and for Arch 3, 31 $\rightarrow$ 20 $\rightarrow$ 24. Furthermore, validation trajectories become noticeably noisier under higher dropout (for Arch 2 at $p = 0.5$, validation accuracy temporarily dips to $\approx 60\%$ around epoch 25 before recovering).

5. **Parameter Efficiency of Architecture 2:** Architecture 1 concentrates 96.8% of its parameters in FC1. In contrast, Architecture 2 contracts the flatten dimension to 288 via its third convolutional stage, requiring only 48,405 parameters. Across all three dropout rates, Architecture 2 achieves the highest test accuracy (80.0%, 83.0%, and 81.0%).

6. **Class Vulnerability Analysis:** Scorpion is the most difficult class for Architecture 1, where test recall remains low across all dropout rates (20% at $p = 0.0$, 25% at $p = 0.3$, and 35% at $p = 0.5$). In Architecture 2, however, scorpion recall increases substantially (70% at $p = 0.0$, 65% at $p = 0.3$, and 75% at $p = 0.5$). In Arch 3, the weakest class at $p = 0.3$ and $p = 0.5$ is starfish (recalls of 55% and 60%), and for Arch 2 at $p = 0.5$ starfish recall is 70%. Thus, while scorpion is the hardest class for Architecture 1, class vulnerability varies across architectures. As a hypothesis, scorpion limbs present thin, high-frequency structures that shallow custom filters struggle to separate from background clutter.

7. **Validation Policy vs. Test Accuracy:** Applying the across-model selection rule on validation metrics selects Arch 1 at $p = 0.3$ (validation accuracy 90.0%, test accuracy 73.0%). With $N = 50$ validation images (2% per image) and $N = 100$ test images (1% per image), the 95% confidence interval for an accuracy around 75% is approximately ± 8.5 percentage points for $N = 100$ and ± 10 percentage points for $N = 50$. Consequently, the observed differences between Arch 1 and Arch 2 fall within statistical sampling variance.

4.3 Part B Q1c: Feature Representations in Best Custom CNN

The best architecture was selected using validation accuracy (Architecture 1 at $p = 0.3$). Figure 4.4 illustrates the post-ReLU feature maps extracted from Conv1 ($8 \times 54 \times 54$) and Conv2 ($16 \times 22 \times 22$) of this optimal model for a single representative training image (*butterfly*).

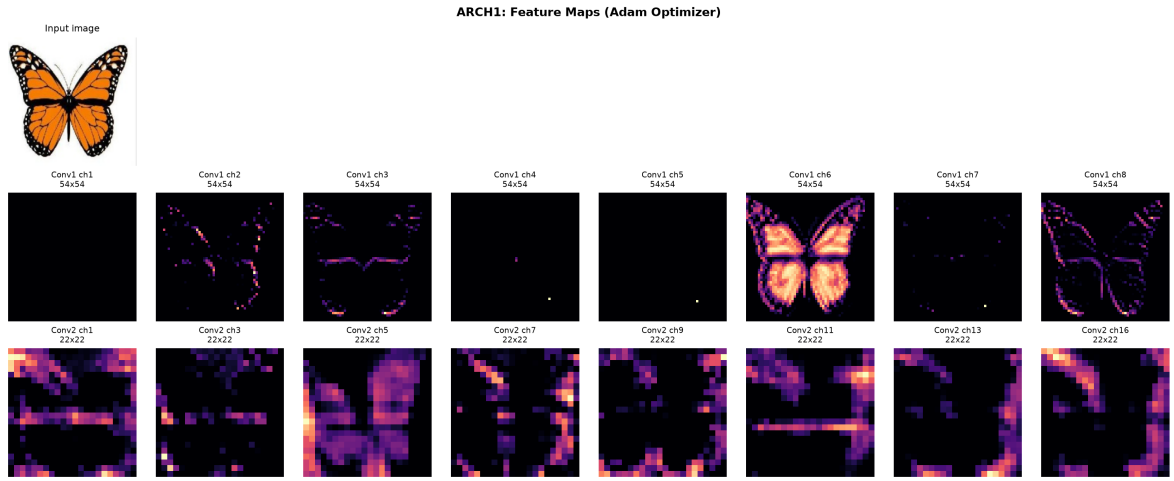


Figure 4.4: Conv1 (54×54) and Conv2 (22×22) post-ReLU feature maps from the optimal Custom CNN (Arch 1, $p = 0.3$) for a representative butterfly training image.

4.3.1 Observation

Observation: For this butterfly training image, all 8 Conv1 feature maps and 8 sampled Conv2 channels (channels 1, 3, 5, 7, 9, 11, 13, 16 in 1-based indexing) are shown. Several Conv1 channels (e.g., channels 1, 4, 5, and 7) are nearly completely inactive (zero activations) for this input, indicating early channel sparsity. Active Conv1 channels capture high-contrast wing margins and spatial boundaries. By Conv2, the feature maps exhibit spatially scattered activations responding to localized sub-patches, responding to localized sub-patches rather than coherent object shapes. We hypothesize that this limited representation capacity contributes to the higher error rate on articulated categories such as scorpion.

4.4 Part B Q1d: Receptive Field Mapping and Bounding Box Analysis

4.4.1 Mathematical Derivation of Receptive Fields

Using the recursive relation in Eq. (1.7)–(1.9), the theoretical receptive field at each layer of the custom architectures is derived:

1. **Input:** $RF_0 = 1, J_0 = 1, start_0 = 0.5$.

2. **Conv1** ($K = 11, S = 4, P = 0$)**:**

$$RF_{\text{conv1}} = 1 + (11 - 1) \cdot 1 = \mathbf{11}, \quad J_{\text{conv1}} = 4, \quad start_{\text{conv1}} = 5.5 \tag{4.2}$$

3. **Pool1** ($K = 3, S = 2$)**:**

$$RF_{\text{pool1}} = 11 + (3 - 1) \cdot 4 = \mathbf{19}, \quad J_{\text{pool1}} = 8, \quad start_{\text{pool1}} = 9.5 \tag{4.3}$$

4. **Conv2** ($K = 5, S = 1, P = 0$)**:**

$$RF_{\text{conv2}} = 19 + (5 - 1) \cdot 8 = \mathbf{51}, \quad J_{\text{conv2}} = 8, \quad start_{\text{conv2}} = 25.5 \tag{4.4}$$

5. **Pool2** ($K = 3, S = 2$)**:**

$$RF_{\text{pool2}} = 51 + (3 - 1) \cdot 8 = \mathbf{67}, \quad J_{\text{pool2}} = 16, \quad start_{\text{pool2}} = 33.5 \tag{4.5}$$

6. **Arch 2 Conv3** ($K = 3, S = 1, P = 0$)**:**

$$RF_{\text{conv3}} = 67 + (3 - 1) \cdot 16 = \mathbf{99}, \quad J_{\text{conv3}} = 16, \quad start_{\text{conv3}} = 49.5 \tag{4.6}$$

The neurons are selected from the output of Conv2 ($16 \times 22 \times 22$) before Pool2, which possesses a theoretical receptive field of $\mathbf{51} \times \mathbf{51}$ pixels with cumulative jump $J = 8$. In Architecture 2, neurons at the output of Conv3 possess an input receptive field of $\mathbf{99} \times \mathbf{99}$ pixels.

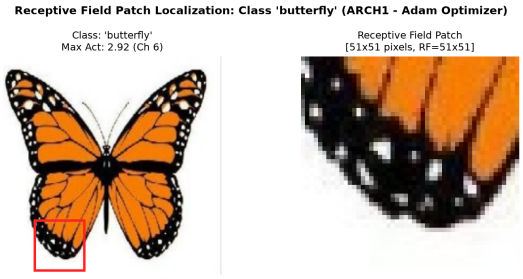
4.4.2 Empirical Receptive Field Patches

For each representative class image, the maximum-activation neuron was identified by computing the global argmax over all channels, rows, and columns of the post-ReLU Conv2 feature map ($16 \times 22 \times 22$). Table 4.4 lists these coordinates, activations, and the resulting 51×51 bounding boxes.

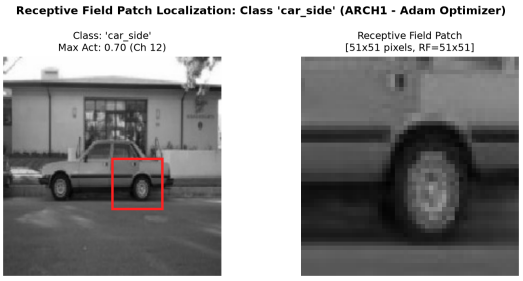
Table 4.4: Maximum-Activation Neurons and 51×51 Receptive Field Coordinates in Optimal Custom CNN (Arch 1, $p = 0.3$).

Category	**Ch (0-idx)**	**Ch (1-idx)**	**Spatial (Row, Col)**	**Max Activation**	**Bounding Box** $[y_1, x_1, y_2, x_2]$
`butterfly`	Ch 5	Ch 6	$(21, 4)$	2.9239	$[168, 32, 219, 83]$
`car_side`	Ch 11	Ch 12	$(13, 14)$	0.6995	$[104, 112, 155, 163]$
`grand_piano`	Ch 3	Ch 4	$(9, 16)$	1.5422	$[72, 128, 123, 179]$
`scorpion`	Ch 5	Ch 6	$(8, 15)$	1.0657	$[64, 120, 115, 171]$
`starfish`	Ch 3	Ch 4	$(14, 8)$	0.5056	$[112, 64, 163, 115]$

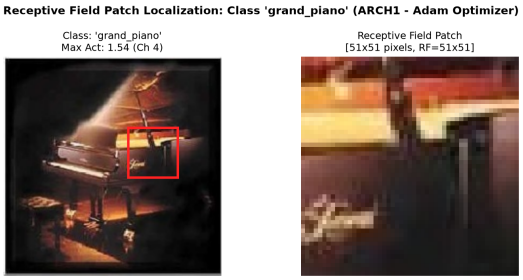
Figure 4.5 depicts these 51×51 bounding boxes mapped onto the Caltech-101 input images, stacked vertically and enlarged for clear visual inspection.



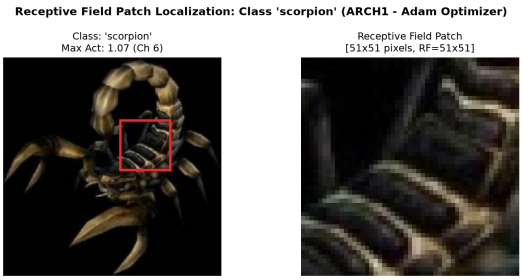
(a) Butterfly



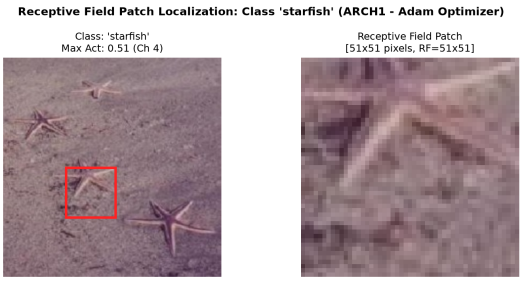
(b) Car Side



(c) Grand Piano



(d) Scorpion



(e) Starfish

Figure 4.5: Receptive field bounding boxes (51×51 pixels) for maximum-activation neurons in the optimal Custom CNN.

4.4.3 Observation

Observation: A 51×51 patch covers $51^2/224^2 \approx 5.2\%$ of the image area. In the butterfly image, the patch captures the lower-left wing edge containing white spot markers. In the car image, it localizes over the wheel and lower body region. In the grand piano, it centers on the lid and keyboard junction. In the scorpion, it captures body segments against the background. In the starfish, it covers one arm rather than the central disc. Because a 51×51 window covers only a localized sub-region, the network lacks whole-object context.

Chapter 5

Part B (Q2): Transfer Learning with VGG-19 and Explainability

To address the data scarcity bottleneck observed in Section 4.2, Part B (Q2) investigates transfer learning using a pre-trained VGG-19 architecture. We evaluate classification performance, determine deep receptive fields, and apply Guided Backpropagation and Grad-CAM to interpret model representations.

5.1 Part B Q2a: Transfer Learning with Pretrained VGG-19

5.1.1 Network Architecture and Feature Extraction

The model was constructed using Keras with pre-trained ImageNet weights from the official VGG-19 implementation, without batch-level normalization. All 16 convolutional layers and dense layers `fc1` and `fc2` were frozen:

$$\frac{\partial \mathcal{L}}{\partial \theta_{\text{backbone}}} = 0 \tag{5.1}$$

A new classification layer `Dense(5)` was attached to the 4096-dimensional `fc2` feature representation, resulting in exactly $\mathbf{20,485}$ trainable parameters $(4,096 \times 5 + 5)$. The loss formulation used sparse categorical cross-entropy with logits.

5.1.2 Training Protocol and Early Stopping

Training used the Adam optimizer with learning rate $\eta = 10^{-3}$, without weight decay or dropout, batch size 16, for up to 30 epochs. Early stopping monitored validation loss with patience 8 (`restore_best_weights=True`). Because pre-trained representations are already informative, the model attained its lowest validation loss at **Epoch 1** (≈ 0.026). Training terminated at Epoch 9, restoring the optimal Epoch 1 checkpoint.

5.1.3 Classification Performance and Error Analysis

Table 5.1 summarizes the performance metrics across data partitions.

Table 5.1: Classification Performance of Pre-trained VGG-19 Feature Extractor on Caltech-101 Subset.

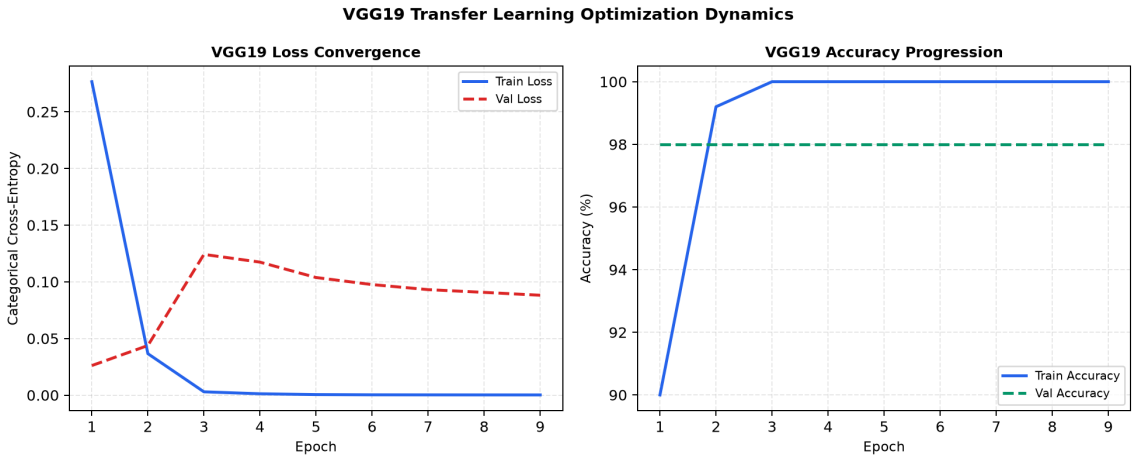
Data Partition	**Validation Loss**[1]	**Accuracy (%)**	**Total Samples**
Training Set	—	98.80%	250
Validation Set	0.026	98.00%	50
Test Set	—	**99.00%**	**100**

The plotted training accuracy at epoch 1 ($\approx 90\%$) in the learning curves reflects a running average during early mini-batches, whereas the reported 98.8% training accuracy is evaluated in inference mode on the restored checkpoint. Out of 100 test images, VGG-19 commits only 1 error (19/20 on *butterfly*, misclassifying one as *starfish*), achieving 20/20 (100%) accuracy on all other four categories (including 0 errors on *scorpion*). Across the other splits:

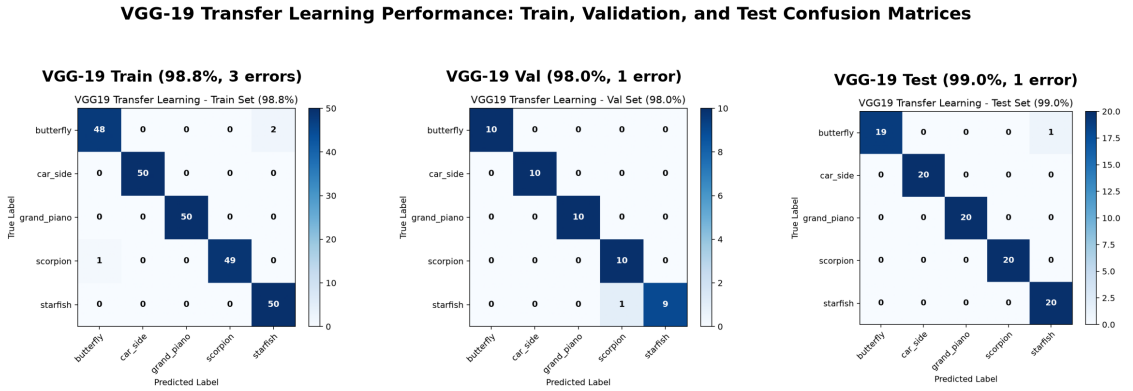
- **Training Errors (3/250):** 2 butterfly $\rightarrow$ starfish; 1 scorpion $\rightarrow$ butterfly.
- **Validation Errors (1/50):** 1 starfish $\rightarrow$ scorpion.
- **Test Errors (1/100):** 1 butterfly $\rightarrow$ starfish.

Figure 5.1 illustrates the training curves and confusion matrices for VGG-19 across all splits. Figure 5.2 displays the side-by-side test confusion matrix comparison between the best custom CNN (Arch 1, $p = 0.3$, 73.0%) and VGG-19 (99.0%).

[1] Validation loss at Epoch 1 read from training trajectory curves (from the VGG-19 training trajectory curves).



(a) VGG-19 Training and Validation Curves



(b) VGG-19 Train, Val, and Test Confusion Matrices

Figure 5.1: Training curves and comprehensive confusion matrix evaluation for VGG-19 feature extraction.

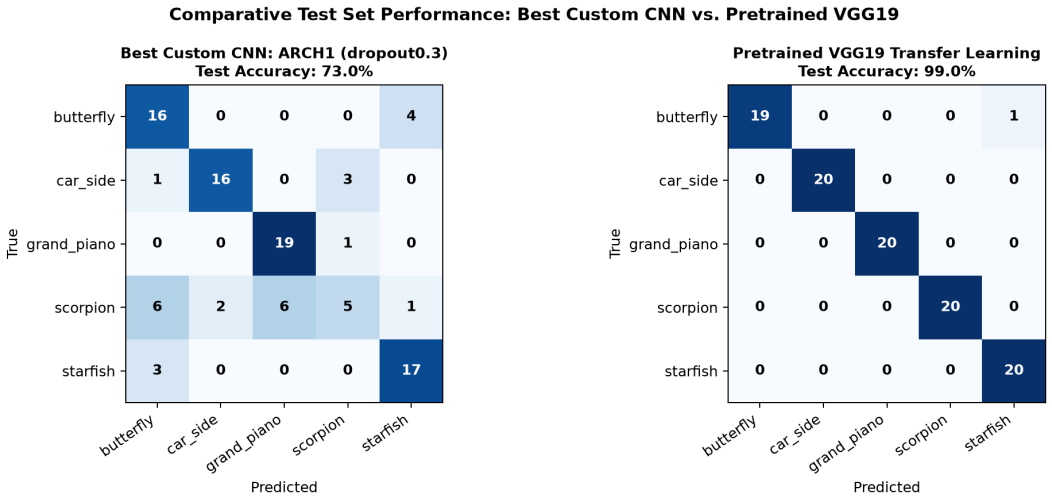


Figure 5.2: Side-by-side comparison of test confusion matrices: Best Custom CNN (Arch 1, $p = 0.3$, 73.0%) vs. Pre-trained VGG-19 (99.0%).

5.1.4 Observation

Observation: Feature extraction with pre-trained VGG-19 provides a substantial improvement over training custom CNNs from scratch (test accuracy increases from 73.0% to **99.0%**, with only a single test error). While Architecture 1 at $p = 0.3$ made 15 errors on scorpion, VGG-19 correctly classifies all 20 test scorpions. We note that ImageNet contains categories closely related to the Caltech-101 subset (e.g., butterflies, scorpions, starfish, and grand pianos), which likely provides pre-trained visual representations well aligned with these categories.

5.2 Part B Q2b: Deep Receptive Field Analysis at `block5_conv4`

5.2.1 Theoretical Derivation at `block5_conv4`

VGG-19 consists of five convolutional blocks separated by max-pooling layers ($2 \times 2, S = 2$), with all conv filters having $K = 3, S = 1, P = 1$. Computing the receptive field recursively through `block5_conv4` (before `block5_pool`, output shape $14 \times 14 \times 512$) yields:

$$J_{\text{block5}} = 2 \times 2 \times 2 \times 2 = 16 \tag{5.2}$$

Accumulating through all 16 convolutional layers and 4 pooling stages yields a theoretical receptive field of:

$$RF_{\text{block5_conv4}} = \mathbf{252} \times \mathbf{252} \text{ pixels} \tag{5.3}$$

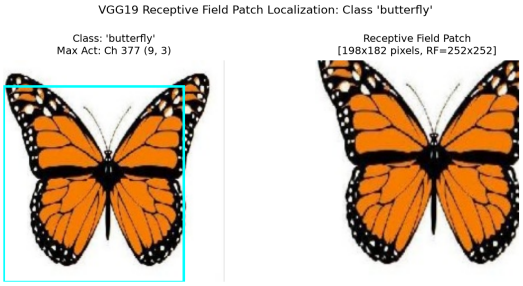
Because the input images are 224×224, the theoretical receptive field of 252×252 exceeds the input dimensions. When clipped to the image boundary, the nominal bounding box covers the entire image. However, the effective receptive field (the region with non-negligible gradient magnitude) is smaller than the theoretical bound; visual attribution methods (Guided Backprop and Grad-CAM) are used below to determine exact spatial sensitivity.

5.2.2 Empirical Receptive Field Coordinates

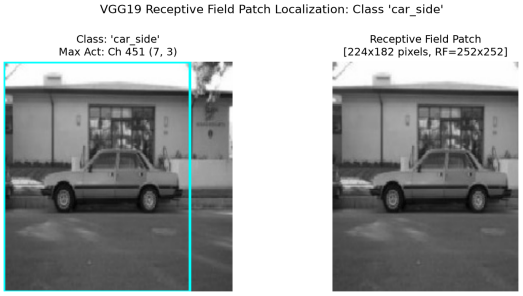
Table 5.2 lists the maximum-activation neurons at `block5_conv4` and their clipped bounding boxes. Figure 5.3 depicts these bounding boxes on the Caltech-101 test samples, stacked vertically at an enlarged scale.

Table 5.2: Maximum-Activation Neurons and Bounding Boxes at `block5_conv4` in VGG-19.

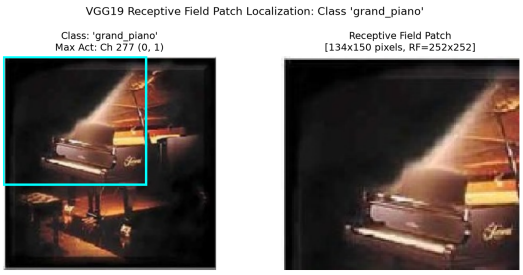
Category	**Ch (0-idx)**	**Ch (1-idx)**	**Row**	**Col**	**Activation**	**Clipped BBox** $[y_1, x_1, y_2, x_2]$
`butterfly`	Ch 376	Ch 377	9	3	368.17	$[26, 0, 224, 182]$
`car_side`	Ch 450	Ch 451	7	3	137.94	$[0, 0, 224, 182]$
`grand_piano`	Ch 276	Ch 277	0	1	157.08	$[0, 0, 134, 150]$
`scorpion`	Ch 104	Ch 105	5	9	130.39	$[0, 26, 214, 224]$
`starfish`	Ch 498	Ch 499	9	9	59.84	$[26, 26, 224, 224]$



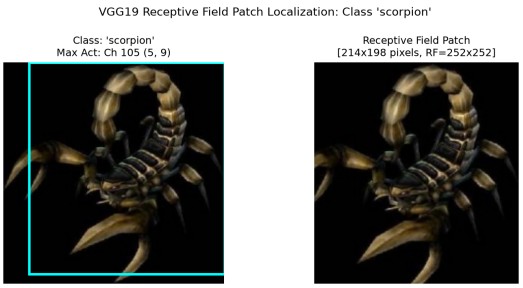
(a) Butterfly



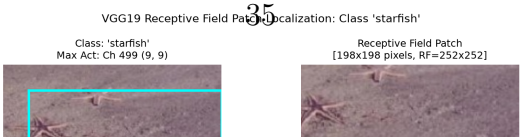
(b) Car Side



(c) Grand Piano



(d) Scorpion



5.3 Part B Q2c: Guided Backpropagation for Feature Attribution

5.3.1 Top-5 Neurons on Butterfly Image

For consistency with the Q1c analysis, Guided Backpropagation was applied to the **same representative butterfly image**, identifying the top-5 neurons across the $14 \times 14 \times 512$ activation tensor of `block5_conv4`. These neurons correspond to the top-5 channels ranked by maximum activation, evaluated at their respective spatial argmax coordinates (row, col), as detailed in Table 5.3.

Table 5.3: Top-5 Activating Neurons at `block5_conv4` on the Representative Butterfly Image.

Rank	Ch (0-idx)	Ch (1-idx)	Row	Col	Activation	Feature Attribution Focus
1	Ch 376	Ch 377	9	3	368.17	Primary wing border and edge curvature
2	Ch 103	Ch 104	4	6	Top-2	Internal wing-vein textures and boundary strokes
3	Ch 373	Ch 374	3	2	Top-3	High-curvature anterior wing margins
4	Ch 385	Ch 386	5	6	Top-4	Central thoracic and wing junction strokes
5	Ch 46	Ch 47	5	6	Top-5	Symmetric abdominal contour strokes

The visualization recipe follows standard practice: the raw input is normalized; gradients are standardized as $(g-\mu)/\sigma \cdot 0.15 + 0.5$; saliency is computed as the maximum absolute gradient across color channels; and the overlay combines $0.35 \times \text{image} + 0.65 \times \text{inferno(saliency)}$.

Figure 5.4 shows the five resulting Guided Backpropagation attributions side by side along with the input butterfly image. Figure 5.5 provides supplementary Guided Backpropagation cards across all five classes.

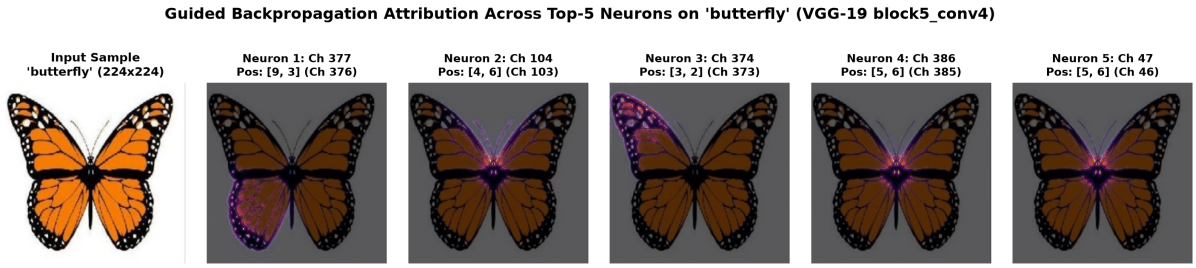
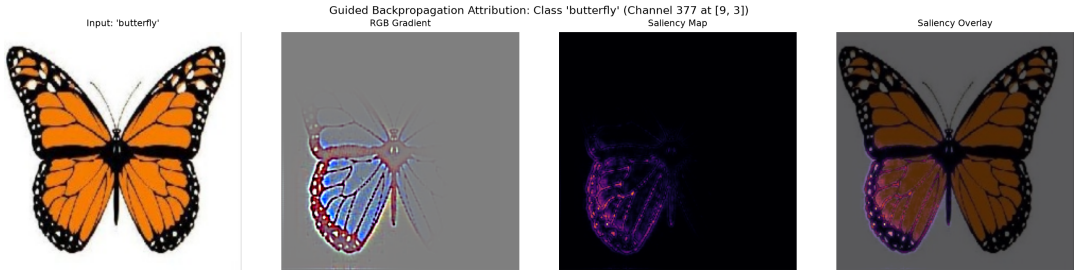
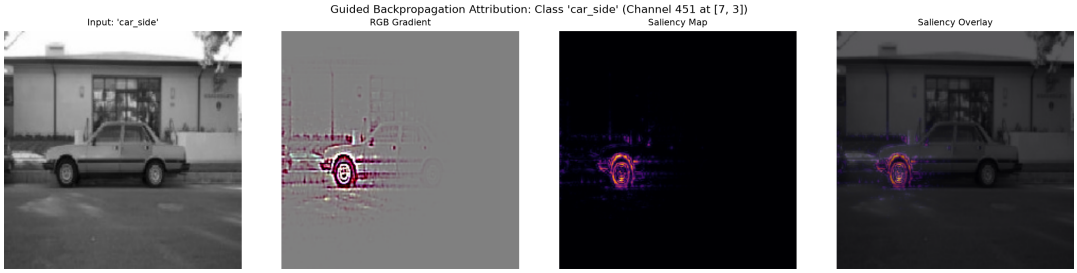


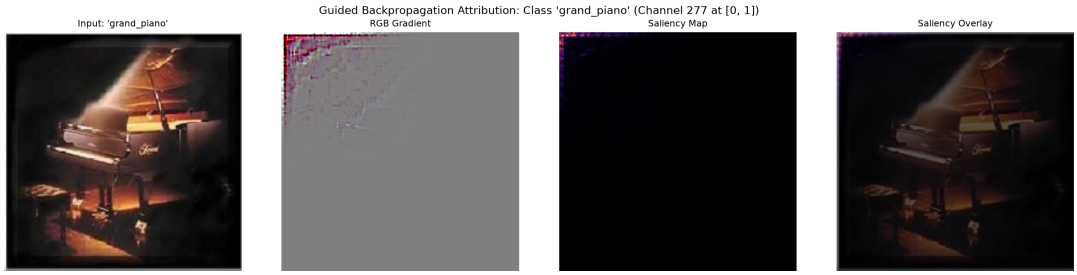
Figure 5.4: Guided Backpropagation feature attribution across the top-5 neurons for the representative butterfly image at `block5_conv4`.



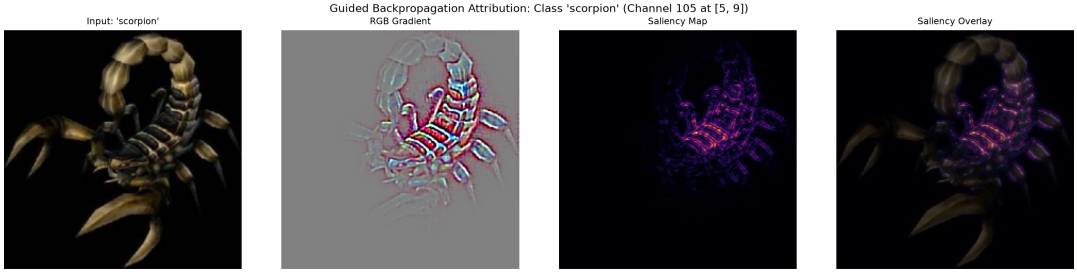
(a) Butterfly



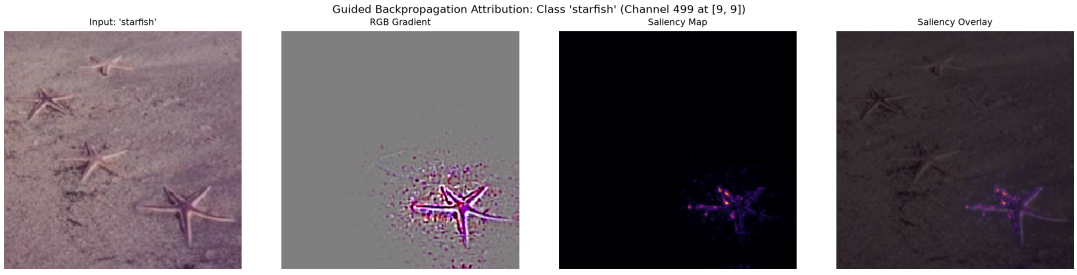
(b) Car Side



(c) Grand Piano



(d) Scorpion



(e) Starfish

Figure 5.5: Supplementary Guided Backpropagation visualizations for the top-activating neuron per class.

5.3.2 Observation

Observation: Guided Backpropagation reveals fine-grained edge templates that stimulate individual deep feature channels. In the butterfly image (which is set against a plain background), Channel 376 highlights the circular margins and edge contours of the wings. In *car_side*, Channel 450 highlights circular boundaries around the wheels and lower metallic contours. In *grand_piano*, the top neuron sits at coordinate $(0, 1)$ in the upper-left corner; its saliency map is largely dark, indicating that attribution is weak and localized to the upper boundary. For *scorpion*, Channel 104 highlights segment boundaries along the body and appendages. For *starfish*, Channel 498 highlights radial arm edges. These visualizations suggest that deep convolutional channels in pre-trained backbones develop selectivity for specific geometric contours.

5.4 Part B Q2d: Class Activation Mapping with Grad-CAM

5.4.1 Full 5×5 Cross-Class Query Matrix

To evaluate class-discriminative localization, Grad-CAM was executed across all 25 combinations (5 input images $\times$ 5 target classes). Gradients of pre-softmax class logits were computed with respect to feature maps at `block5_conv4`, globally average-pooled to obtain channel weights α_k^c, combined linearly, passed through a ReLU, normalized by the maximum activation, and bilinearly upsampled to 224×224. Figure 5.6 displays the complete 5×5 query matrix.

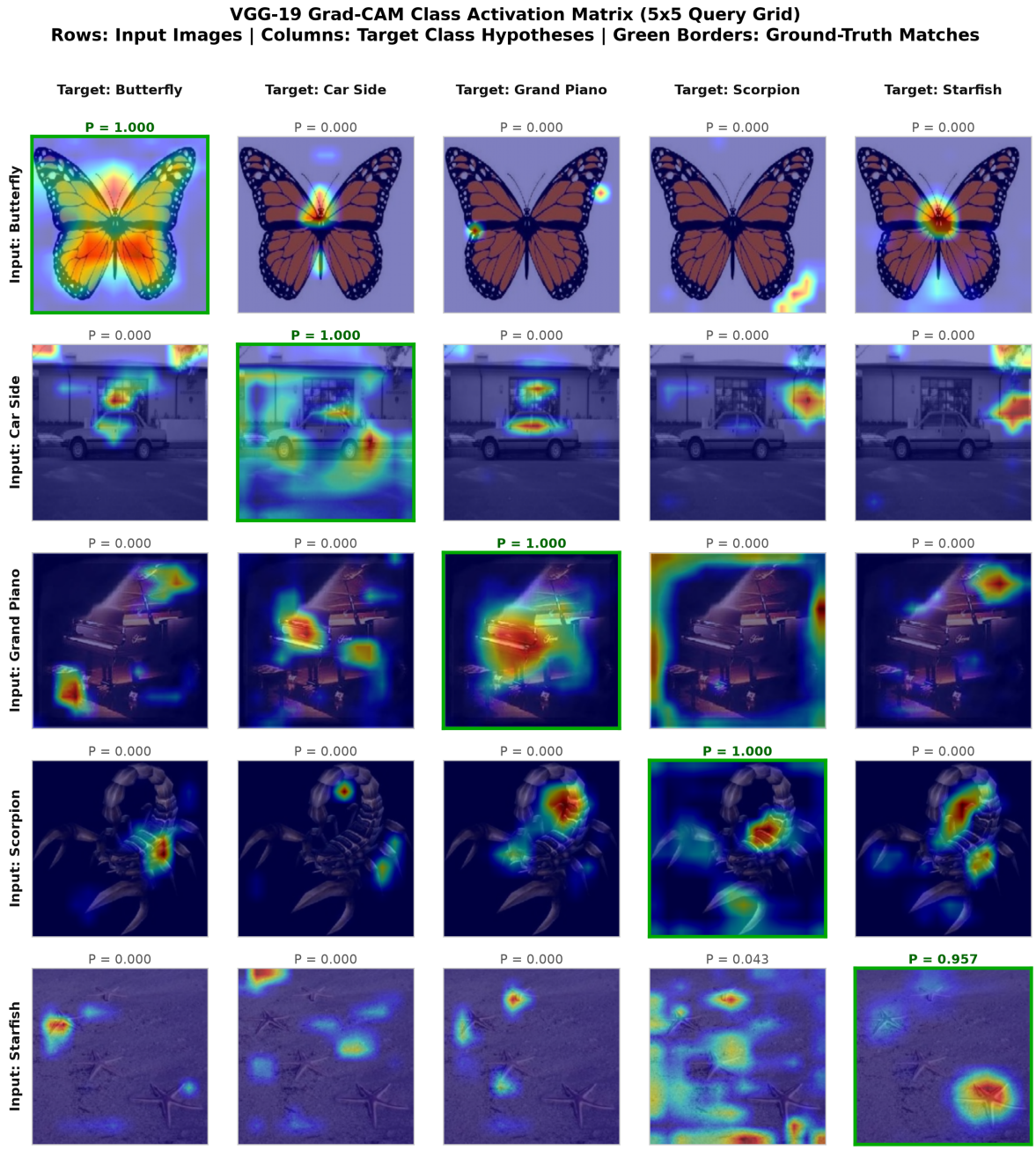
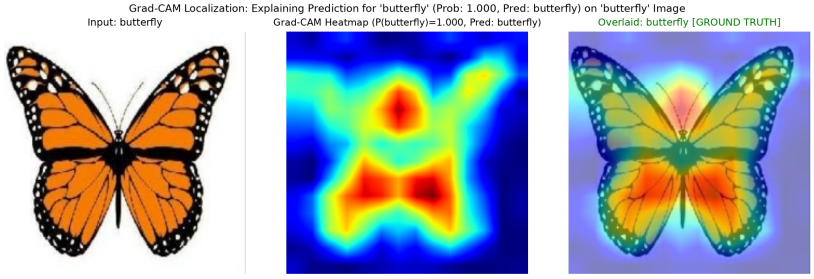
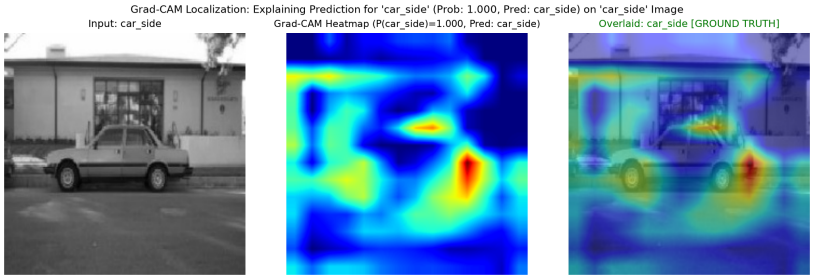


Figure 5.6: Complete 5×5 Grad-CAM localization matrix: Rows correspond to input images, columns to queried target classes, with Softmax probabilities indicated above each panel.

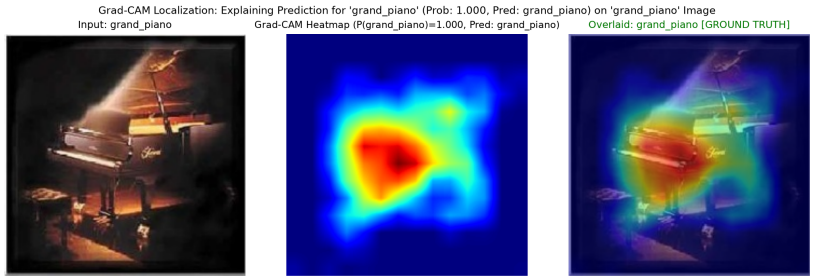
Figure 5.7 provides enlarged views of the on-target explanations (ground truth matching prediction), while Figure 5.8 illustrates counterfactual explanations for the butterfly input.



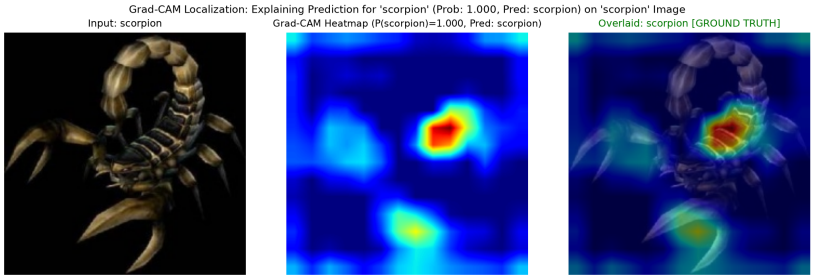
(a) Butterfly ($p = 1.000$)



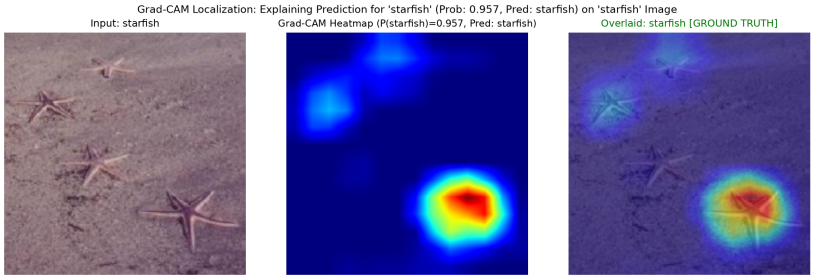
(b) Car Side ($p = 1.000$)



(c) Grand Piano ($p = 1.000$)

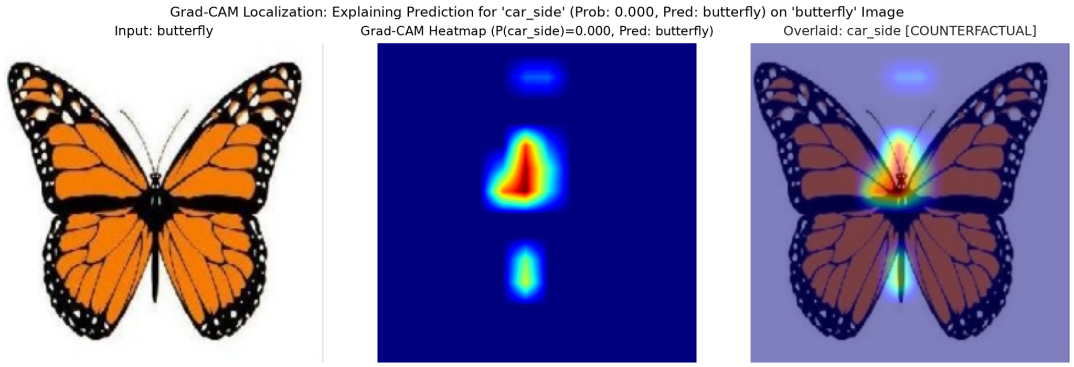


(d) Scorpion ($p = 1.000$)

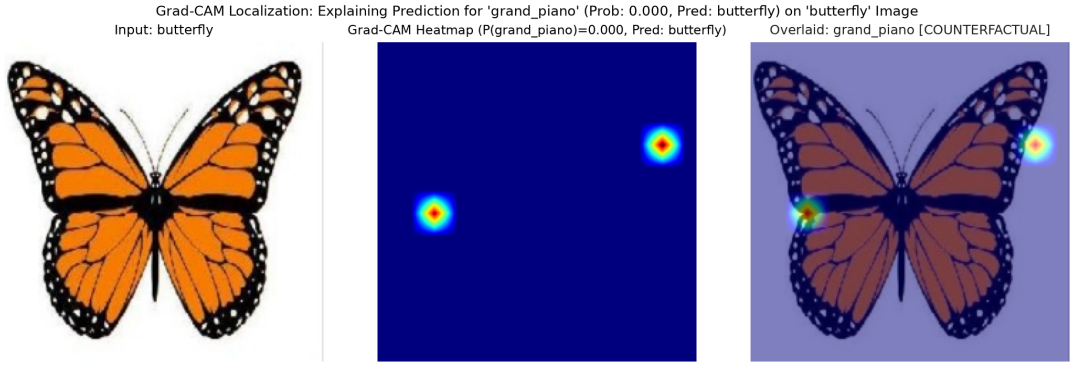


(e) Starfish ($p = 0.957$)

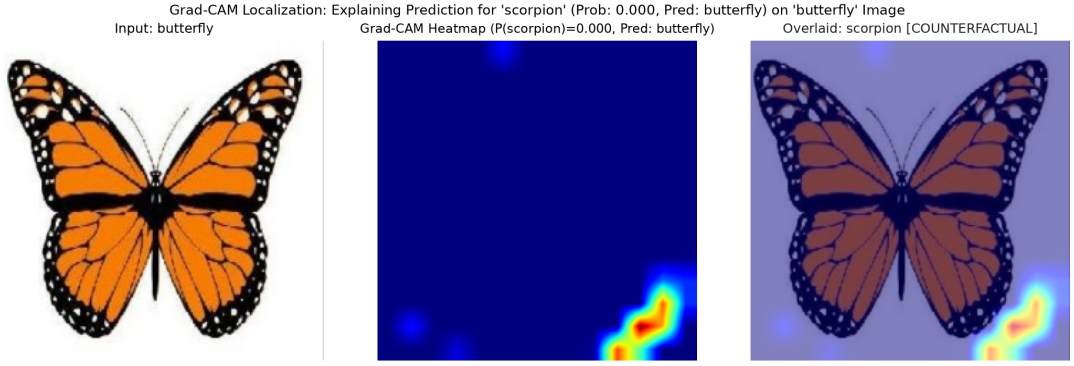
Figure 5.7: Enlarged on-target Grad-CAM heatmaps: Target class matches ground truth ($p \geq 0.999$ for four classes, 0.957 for starfish).



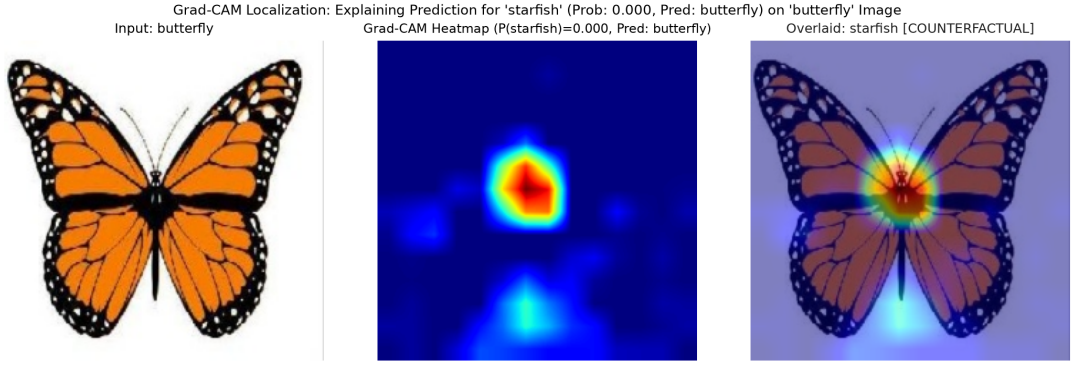
(a) Target: Car ($p = 0.000$)



(b) Target: Piano ($p = 0.000$)



(c) Target: Scorpion ($p = 0.000$)



(d) Target: Starfish ($p = 0.000$)

Figure 5.8: Counterfactual Grad-CAM heatmaps evaluating a butterfly input against non-target class hypotheses.

5.4.2 Observation

Observation: For on-target queries, Grad-CAM heatmaps concentrate on foreground regions: the wing patterns of the butterfly, the chassis and wheel areas of the car, the body of the grand piano, the segmented anatomy of the scorpion, and the radial limbs of the starfish. In four of the five ground-truth instances, the predicted Softmax probability is ≥ 0.999, while for starfish it is 0.957.

Because every Grad-CAM map is normalized by dividing by its maximum activation, counterfactual heatmaps also exhibit peak intensities of 1.0 (appearing as localized hotspots rather than broad, low-amplitude patterns). The primary evidence against incorrect class hypotheses is reflected in the Softmax probabilities, which drop to 0.000 across all 20 counterfactual pairs (except for starfish queried as scorpion, which receives $p = 0.043$). This suggests that while feature maps can identify weak local correlations under forced target queries, the classification layer suppresses these hypotheses in its final posterior distribution.

Chapter 6

Synthesis, Comparative Discussion, and Conclusion

This chapter synthesizes the core findings from Parts A and B, examining the interaction between architectural priors, regularization, dataset scale, and visual interpretability.

6.1 Synthesis of Findings

1. **Architectural Inductive Biases vs. Dataset Scale:** In Part A, training on 11,385 MNIST images enabled a two-stage CNN to achieve **98.02%** test accuracy with minimal overfitting (generalization gap 0.87 percentage points). In Part B, however, training custom CNNs from scratch on only 250 Caltech-101 images resulted in marked overfitting across all nine configurations (training accuracies 91.6%–100.0%, but test accuracies restricted to 62.0%–83.0%). CNNs possess structural inductive biases (weight sharing, sparse connectivity), but without sufficient training samples or data augmentation, these priors alone cannot prevent over-parameterized dense layers from memorizing small training sets.

2. **Regularization and Depth Dynamics:** Dropout regularized the networks on average, contracting the mean generalization gap from 18.4 to 11.5 percentage points across the 9-model sweep. The deepest architecture (Arch 3) benefited most in test accuracy (62.0% $\rightarrow$ 78.0%). However, increasing depth provided no inherent advantage under small sample sizes: Arch 3 consistently lagged behind the more compact Arch 2. Furthermore, Architecture 1 concentrated 96.8% of its parameters into FC1. The parameter-efficient Architecture 2 (48,405 parameters) achieved the best test accuracy at every dropout level (80.0%, 83.0%, and 81.0%).

3. **The Impact of Transfer Learning:** Feature extraction with pre-trained VGG-19 largely removed the data scarcity bottleneck, lifting test accuracy from 73.0% to **99.0%** with a single test error. Transfer learning enables target tasks to benefit from generic edge,

texture, and shape representations pre-trained on ImageNet, requiring only adaptation of final classification weights.

4. **Complementarity of Explainability Techniques:** Receptive field analysis establishes theoretical spatial bounds (14×14 in Part A, 51×51 in Arch 1, 99×99 in Arch 2, and 252×252 in VGG-19). Guided Backpropagation provides high-resolution pixel attributions showing which edge patterns excite a neuron, while Grad-CAM delivers class-discriminative localization heatmaps revealing where the network attends for final decisions. Used together, they offer complementary views of model representation and decision logic.

6.2 Study Limitations

We note several limitations in the experimental setup:

- **Single Deterministic Run:** All empirical results reflect a single run under seed 42. Multi-seed training with mean and standard deviation reporting would be required to establish statistical significance.
- **Dataset Scale:** The validation set consists of 50 images (2% per image) and the test set consists of 100 images (1% per image). Discrepancies within ±8–10 percentage points remain within expected sampling noise.
- **Absence of Data Augmentation:** Reported custom CNN models did not utilize geometric or photometric data augmentation, which would likely reduce the generalization gap.
- **Semantic Overlap:** The five chosen Caltech-101 categories share semantic proximity with ImageNet classes, which facilitates transfer learning compared to out-of-domain transfer tasks.

6.3 Concluding Remarks

This study evaluated convolutional architectures across synthetic and natural visual domains. Through systematic experimentation, we examined spatial filtering, evaluated dropout regularization under small sample sizes, confirmed the utility of pre-trained feature extraction, and applied gradient attribution techniques to analyze learned representations.

References

[1] Dileep A.D., *Lecture Slides 22–26: Convolutional Neural Networks*, Course CS601T: Deep Learning, Department of Computer Science and Engineering, Indian Institute of Technology Dharwad, 2026.

[2] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition", *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[3] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks", *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 25, pp. 1097–1105, 2012.

[4] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition", *arXiv preprint arXiv:1409.1556*, 2014 (Published at ICLR 2015).

[5] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification", *IEEE International Conference on Computer Vision (ICCV)*, pp. 1026–1034, 2015.

[6] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks", *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 249–256, 2010.

[7] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting", *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.

[8] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization", *arXiv preprint arXiv:1412.6980*, 2014 (Published at ICLR 2015).

[9] J. T. Springenberg, A. Dosovitskiy, T. Brox, and M. Riedmiller, "Striving for simplicity: The all convolutional net", *arXiv preprint arXiv:1412.6806*, 2014 (Published at ICLR Workshop 2015).

[10] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization", *IEEE International Conference on Computer Vision (ICCV)*, pp. 618–626, 2017.

[11] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, Cambridge, MA, 2016. http://www.deeplearningbook.org.

[12] L. Fei-Fei, R. Fergus, and P. Perona, "Learning generative visual models from few training examples: An incremental Bayesian approach tested on 101 object categories", *IEEE CVPR Workshop on Generative-Model Based Vision*, 2004.

[13] Y. LeCun, C. Cortes, and C. J. Burges, "The MNIST database of handwritten digits", 1998. http://yann.lecun.com/exdb/mnist/.

[14] F. Chollet, "Grad-CAM class activation visualization", *Keras Code Examples*, 2020. https://keras.io/examples/vision/grad_cam/.